

ZeBu[®] Virtual Device Model Solution User Guide

Version W-2025.06, September 2025



Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

About This Book	6
Intended Audience	6
Contents of This Book	6
Related Documentation	7
Additional Resources	7
Typographical Conventions	7
Synopsys Statement on Inclusivity and Diversity	8

1. Introducing Virtual Device Model Solution	9
Overview	9
Benefits of the Solution	11
Requirements	12
Software and Hardware Requirements	12
Supported and Unsupported VS Versions	13
License Information	14

2. Installation and Setup	15
Install the Virtual Device Model Solution	15
Directory Structure	16
Set Up the User Environment	16
Set Up the Verification Environment	16
Standalone Virtual Device Model Solution Environment	17
Compile the Testbench	18

3. PCIe	19
Supported PCIe Topologies	20
Single-Function PCIe Topology	20
Arbitrary Topology	21
ARI Topology	22
Supported PCIe Functions	23

Contents

Using the Test Functions	23
Simple Function	24
NVMe Test Function	26
BSA Test Function	31
ATS/PRI Test Function	32
User-Defined Functions	39
Data Object Exchange (DOE)	40
Integrity and Data Encryption (IDE)	42
Pre-Shared Keys	43
IDE_KM Over SPDM	46
IDE_KM Over DOE	48
Key Swapping	49
TEE Device Interface Security Protocol (TDISP)	49
Configuring the Physical Layer	52
Maximum Payload Size	52
Maximum Link Speed	52
Cycle Accurate Mode	53
Control Dynamic Lane Width	54
Enabling PCIe Power Management	54
D-State Transitions	55
L-State Transitions	55
Advanced Error Reporting (AER)	55
Enabling PCIe or USB Transactor in ProxyEnv	56
Support for 10-bit Tag	56
Support for 14-bit Tag	57
PCIe Debug Verbosity	57
Monitoring Performance	58
Enabling PCIe Protocol Analyzer From VDM	58
In-Band Commands	59
4. CXL	61
Single-Device CXL Topology	62
CXL Type 1 Function	62
CXL Type 3 Function	63
CXL Memory Accesses	64
CXL Memory Device Delay Modeling	64
Predefined Response Delay Model Functions	65
User-Defined Response Delay Model Functions	65

Contents

CXL 2.0 Type 1	68
Integrity and Data Encryption for CXL 2.0 Gen5	68
CXL Hot Plug	69
CXL Hot Reset	70
Flit Mode	70
5. USB	72
6. Examples	73
Settings for Running an Example	73
PCIe Example	74
CXL Example	75
USB Example	76
7. Appendix: Verified Software Drivers	78
PCIe Software Driver	78
CXL Software Driver	80
DOE Software Driver	81
NIMG File Format	81

Preface

This chapter has the following sections:

- [About This Book](#)
- [Intended Audience](#)
- [Contents of This Book](#)
- [Related Documentation](#)
- [Additional Resources](#)
- [Typographical Conventions](#)
- [Synopsys Statement on Inclusivity and Diversity](#)

About This Book

The *ZeBu® Virtual Device Model Solution User Guide* introduces the solution, the installation and setup procedure of the solution, and how to create virtual PCIe, USB, and CXL devices using the solution.

Intended Audience

The content is for engineers who need a virtual device, such as PCIe, to verify the PCIe host system using simulation, simulation acceleration, and emulation. The engineers should have basic to advanced knowledge of using the following products: PCI Express, NVMe, CXL EP, USB, Simulation Acceleration, and the ZeBu platform.

Contents of This Book

The *ZeBu Virtual Device Model Solution User Guide* has the following chapters:

Chapter	Describes...
Introducing Virtual Device Model Solution	Describes the solution and its benefits. It also lists the hardware and software requirements for installing and setting up the solution.
Installation and Setup	Lists the steps to install and setup the solution.

Chapter	Describes...
Modeling a PCIe Topology	Describes the supported PCIe topologies and the functions (predefined and user-defined) that can be used to model it.
Modeling the CXL Topology	Describes the supported CXL topologies and the functions (predefined and user-defined) that can be used to model it.
USB support in VDM	Lists the USB speeds supported by VDM.
Appendix: Verified Software Drivers	Provides information about the verified software drivers.

Related Documentation

Document Name	Description
<i>ZeBu Virtual Host Solutions User Manual</i>	Provides usage of Virtual Host Solution with various ZeBu transactors.
ZeBu BSA Compliance Testing - Reference Guide	Describes pre-silicon ARM BSA compliance testing, its benefits, and usage using the BSA add-on for the Zebu Virtual Devices Solution.

Additional Resources

- PCI Express Base Specification, Revision 6.0
- NVM Expression, Revision 1.3
- Compute Express Link, Revision 2.0
- Security Protocol and Data Model Specification, DSP0274, Version 1.2
- Secured Messages Using SPDm Specification, DSP0277, Version 1.0.0

Typographical Conventions

This document uses the following typographical conventions:

To indicate	Convention Used
Program code	<i>OUT</i> <= <i>IN</i> ;

To indicate	Convention Used
Object names	<i>OUT</i>
Variables representing objects names	<i><sig-name></i>
Message	Active low signal name ' <i><sig-name></i> ' must end with <i>_X</i> .
Message location	<i>OUT</i> <= IN;
Reworked example with message removed	<i>OUT_X</i> <= IN;
Important Information	NOTE: This rule...

The following table describes the syntax used in this document:

Syntax	Description
[] (Square brackets)	An optional entry
{ } (Curly braces)	An entry that can be specified once or multiple times
(Vertical bar)	A list of choices out of which you can choose one
... (Horizontal ellipsis)	Other options that you can specify

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introducing Virtual Device Model Solution

VDM provides a collection of easy-to-use components terminating various physical interfaces. It supports multiple instances of one or more physical interfaces and one or more endpoint devices. An emulated host design might have multiple interfaces of the same or different types, all connected to suitable reactive endpoints provided by VDM. This document describes the use of a single interface and a single reactive endpoint, they can be combined to suit the need of the verification environment.

For more information, see the following sections:

- [Overview](#)
- [Benefits of the Solution](#)
- [Requirements](#)

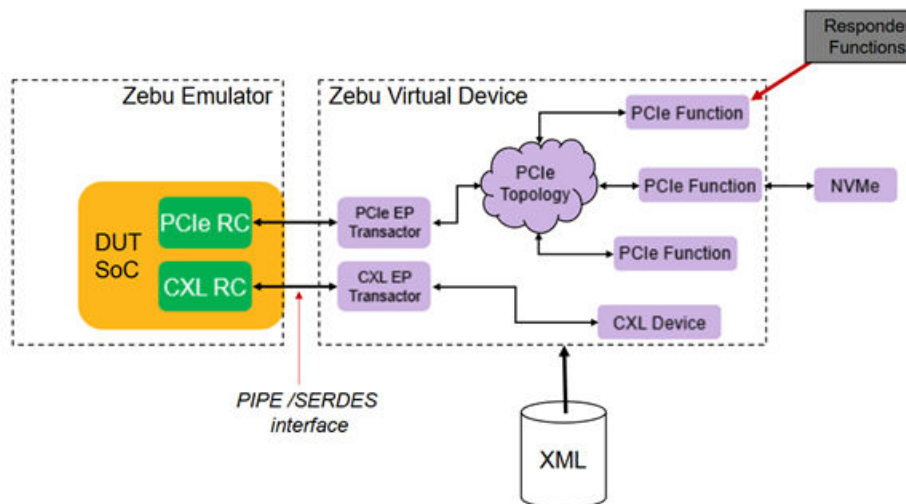
Overview

VDM Architecture

Virtual Device Model Solution (VDM) provides a suitable reactive endpoint connected to a traffic-initiating physical interface in a host design emulated in ZeBu. VDM emulates the physical connection to various peripheral devices (as if the connections were directly connected to the design under verification). It supports multiple physical interfaces, and

each interface might support multiple target device types. This is shown in the following diagram:

Figure 1 Virtual Device Model Solution Topology for PCIe and CXL



Features

VDM has the following features:

- It is an easy-to-use solution for terminating a PCI Express (PCIe) link and providing a variety of devices and topologies. It can model an arbitrary PCIe topology, with multiple devices and functions, responding to downstream configuration and memory requests. The solution can also initiate upstream memory requests of their own. It provides several predefined PCIe functions that can be instantiated to create a suitable endpoint for verification objectives.
 - PCIe Bridge: Used to create arbitrary PCIe topologies, including ARI support
 - MMIO Function: Simple function with MMIO regions that can be used as a target for memory access.
 - NVMe Function: Used to emulate an NVMe drive, responding to configuration and data access commands. The drive data can be initialized from an ISO file.
 - BSA Tester Function: Provides suitable responder and initiator capabilities for executing the ARM BSA compliance test suite.
- It provides an easy-to-use solution for terminating a Compute Express Link (CXL) and a variety of reactive device models. Built on top of the PCI Express solution, it can respond to CXL.io requests, as well as respond and initiate CXL.mem and CXL.cache requests. In addition, it provides several predefined CXL functions that can be instantiated to create a suitable endpoint for verification objectives.

- CXL Type 1: Simple function with CXL cache
- CXL Type 3: Simple function with CXL memory ranges and Host-managed Device Memory, with CDAT support.

Note:

CXL topologies are not yet supported.

- It provides an easy-to-use NVMe function. It emulates a real NVMe 1.3 SSD drive responding to configuration and data access commands as required by the standard Linux or Windows 10 NVMe device driver. To use it, you specify the initial configuration parameters only. The rest of the process is automatic. The transactor saves the data written during an emulation run to a file, which you can reuse in a subsequent emulation run, making the previously written data available.

Multiple instances of the NVMe function can be instantiated in an arbitrary PCIe topology.

- The USB Virtual Device Model is a mass storage device that implements the Bulk Only Transport protocol. It supports following SCSI commands:
 - INQUIRY
 - TEST_UNIT_READY
 - REQUEST_SENSE
 - MODE_SENSE
 - READ_FORMAT_CAPACITY
 - READ_CAPACITY
 - READ
 - WRITE

Benefits of the Solution

VDM provides the following benefits:

- Requires few configuration settings beyond the creation of the C++ testbench
- A single instance of the C++ VDM manager object handles all the interfaces and topologies, using a runtime XML configuration
- Handles all ZeBu transactor initialization and runtime inputs and outputs
- Real-time emulation at MHz speeds

- Post pre-silicon hardware IP (Intellectual Property) integration
- Complex advanced hardware debugging
- Allows software development enablement
- Enables device driver development and validation in DUT
- Easy-to-use solution with minimal C++ testbench code changes
- All transactor configurations are handled through XML

Requirements

The following topics describe the requirements to run VDM:

- [Software and Hardware Requirements](#)
- [Supported and Unsupported VS Versions](#)
- [License Information](#)

Software and Hardware Requirements

VDM has the following software and hardware requirements. For specific version compatibility information, see the *ZeBu[®] Virtual System Adaptor Release Notes*.

- ZeBu emulation hardware and software
- ZeBu Vertical Solutions transactors:
 - PCIE DPI Transactor
 - CXL DPI Transactor
 - USB3 DPI Transactor

Table 1 *Operating Systems in the QSC*

Platform	Operating System	Compatible versions
Linux 32 bits	CentOS 7.3.1611	CentOS: 7.3.1611+ RHEL: 7.3+; 8+ SLES: 12-SP5+; 15-SP2+ Rocky Linux: 8.4+ AlmaLinux: 8.4+
Linux 64 bits	CentOS 7.3.1611	CentOS: 7.3.1611+ RHEL: 7.3+; 8+ SLES: 12-SP5+; 15-SP2+ Rocky Linux: 8.4+ AlmaLinux: 8.4+

Supported and Unsupported VS Versions

VSA solutions are compatible with the W-2025.06 release of ZeBu Vertical Solutions.

For every version of VSA, there are supported combinations of ZeBu and transactors. Any unsupported combinations of these toolsets cause unexpected behavior. The following table lists supported versions of ZeBu and Transactors for each release of VSA.

Table 2 *Supported Versions for ZeBu and Transactors*

VSA Release	ZeBu Version	Transactor Version
W-2025.06	W-2025.06	W-2025.06-BETA3
V-2024.03-SP1	V-2024.03-1-TZ	V-2024.03-SP1
V-2024.03	V-2024.03-1	V-2024.03
S-2023.03-SP1	S-2021.09-2	S-2023.03-SP1
	S-2021.09-1	S-2023.03
	Q-2020.03-SP1-4	Q-2021.06-SP3
	S-2021.09-1	S-2023.03
S-2023.03	Q-2020.03-SP1-4	Q-2021.06-SP3
	Q-2020.03-SP1-3	Q-2021.06-SP2
		Q-2021.06-SP1
S-2023.03-SP2	S-2021.09-2	S-2023.03-SP2

Mixing “S” and “Q” streams of ZeBu and transactor versions is not supported and should not be used. For instance, you cannot use S-2021.09-2 version of ZeBu with Q-2021.06-SP3 version of transactors.

License Information

The following table lists license key information required for VDM:

Table 3 *License Keys*

Component	License Key
PCIe EP	hw_vdm_pcie
PCIe with Gen6	hw_vdm_pcie6
CXL	hw_vdm_cxl
CXL 3.0	hw_vdm_cxl3
IDE with PCIe or CXL	hw_vdm_ide
NVMe	hw_vdm_nvme
PCIe NIC	hw_vdm_nic
USB Storage	hw_vdm_usb_storage
NVMe Analyzer	hw_vsa_nvme_analyzer

2

Installation and Setup

This section describes the installation and setup requirements for using VDM. VDM is distributed as a self-extracting archive script named as *VSA-VDM-<release_version>.sh*. This chapter contains the following subtopics:

- [Install the Virtual Device Model Solution](#)
- [Directory Structure](#)
- [Set Up the User Environment](#)
- [Set Up the Verification Environment](#)
- [Compile the Testbench](#)

Install the Virtual Device Model Solution

After downloading the solution, use the following command to make the file executable, if not already.

```
% chmod u+x VSA-VDM-<release_version>.sh
```

To install VDM, perform the following steps:

1. Select a directory where the solution is to be installed. To install the solution in a new location, first create the directory.
2. Choose a location that is visible on all machines that need to access and run the solution.

```
% mkdir /tools
```

3. Install the solution in the installation directory by using the following command:

```
% ./VSA-VDM-<release_version>.sh install /tools
```

Examples can be found in *\$VSA_VDM_ROOT/Examples* directory.

Directory Structure

After successfully installing the solution, the following directory structure is created:

```
├── setup.csh ..... User environment setup source file
├── bin ..... Utility scripts and commands
├── doc ..... Documentation
│   ├── html ..... Detailed C++ API documentation
├── Examples ..... User examples
├── include/virtDev ..... Include files
├── libABI1/ ..... Shared library, using C++-11 ABI
```

```
├── drivers
│   ├── include
│   ├── libABI1/
│   │   ├── aarch64/
│   │   └── x86/
├── sled
├── ZEBU_IP_ROOT
│   ├── V-2024.03-SP1
│   │   ├── lib -> libABI1
│   │   └── libABI1
```

Note:

ABI0 libraries are no longer available.

Set Up the User Environment

Each user must add environment variables to use VDM, then source the setup script file. This must be done after the ZeBu tools and ZeBu Vertical Solution transactors have been configured, that is, \$ZEBU_ROOT and \$ZEBU_IP_ROOT are set. To do this, use the following commands:

```
% setenv VSA_VDM_ROOT /tools/VSA-VDM-<release_version>
% source $VSA_VDM_ROOT/setup.csh
```

Set Up the Verification Environment

VDM environment instantiates and manages all of the transactors and virtual devices required by its interfaces. For example, it creates and manages the entire PCIe topology behind a PCIe link. All physical interfaces in the DUT, for example all links from Root Complex instances, can be connected to a single VDM environment instance, each using their own ZeBu transactor instance. It is also possible to have multiple environment instances in the same testbench as well as mixing other transactors not managed by the Virtual Device Environment for VDM.

Standalone Virtual Device Model Solution Environment

The simplest use-model is when all transactors in the ZeBu environment are managed by a single VDM environment. This VDM environment manages the entire interaction with the ZeBu infrastructure and transactors.

The VDM environment is created by instantiating the `SNPS::VSA::VDM::DeviceEnv::Env` class using the simple-form constructor defined in the `$VSA_VDM_ROOT/include/vdm/DeviceEnv/Env.hh` include file as follows:

```
SNPS::VSA::VDM::DeviceEnv::Env MyEnv(const char*   name,
const char*   XMLfile);
```

Name	Name of the Instance
XML file	Path name of the XML file describing the Virtual Device topology

The device topology is specified using an XML file, which can also specify additional options for each element of the topology. A single XML file can specify device topologies for multiple physical interfaces; each connected to their own physical port in the DUT. Example XML configuration files can be found in the `${VSA_VDM_ROOT}/Examples/XML` directory. See `${VSA_VDM_ROOT}/Docs/schema.yaml` for a detailed specification of the XML schema.

After creation, the environment is started by calling the `SNPS::VSA::VDM::DeviceEnv::Env::init()` followed by the `SNPS::VSA::VDM::DeviceEnv::Env::start()` method. To cleanly terminate the emulation run, call `SNPS::VSA::VDM::DeviceEnv::Env::shutdown()` method. After successful shutdown, the environment object may be destroyed.

```
#include "vdm/DeviceEnv/Env.hh"
...
SNPS::VSA::VDM::DeviceEnv::Env myEnv("PCIe Env", "PCIeTopology.xml");
myEnv.init();
...
myEnv.start();
...
myEnv.shutdown();
```

An example of a standalone environment using zRci can be found under the `${VSA_VDM_ROOT}/Examples/StandAlone` directory.

Compile the Testbench

To include VDM in a verification or compliance environment, use the following steps:

- To compile a testbench that uses components from the Virtual Devices Solution, you must add the suggested additional compile-time arguments generated by the '`vsa-vdm-config --cflags`' command:

```
g++ ... `vsa-vdm-config --cflags` ...
```

- To link a testbench that uses components from the Virtual Devices Solution, you must add the suggested additional link-time arguments generated by the '`vsa-vdm-config --ldflags`' command:

```
g++ ... `vsa-vdm-config --ldflags` ...
```

If the testbench makes use of any components in the `SNPS::VSA::VDM::DMTF::SPDM` namespace, the `-llibspdm` command line option must be added at link time. Furthermore, the OpenSSL libraries `libcrypt.so.3` and `libssl.so.3` must be findable via the `LD_LIBRARY_PATH` search path.

3

PCIe

The *Virtual Device Model Solution (VDM)* enables users to create arbitrary topologies of PCIe bridges, devices, and functions connected to an instance of a PCIe Root Complex in a DUT.

The devices that can be modeled as part of this topology include:

- Bridges (including support for peer-to-peer routing within the topology)
- Multifunction devices, including ARI devices

VDM offers four predefined functions that can be used to create PCIe devices. Additionally, you can also use user-defined functions for creating PCIe devices.

The PCIe topology is specified in the XML file, in a `<virtDev::PCIe>` element.

You should correctly enumerate the PCIe topology constructed, appropriately configure bridge bus numbers and memory address apertures, and assign memory resources.

Following are the supported PCIe device models:

- PCIe Bridge
- Simple MMIO function
- ATS/PRI Tester Function
- NVMe SSD
- BSA Tester function
- User-Defined Function
- Intel 82573 compatible NIC Interface
- IDE is supported on the PCIe interface (single EP only)

Supported PCIe Topologies

VDM supports the following PCIe topologies:

- [Single-Function PCIe Topology](#)
- [Arbitrary Topology](#)
- [ARI Topology](#)

Single-Function PCIe Topology

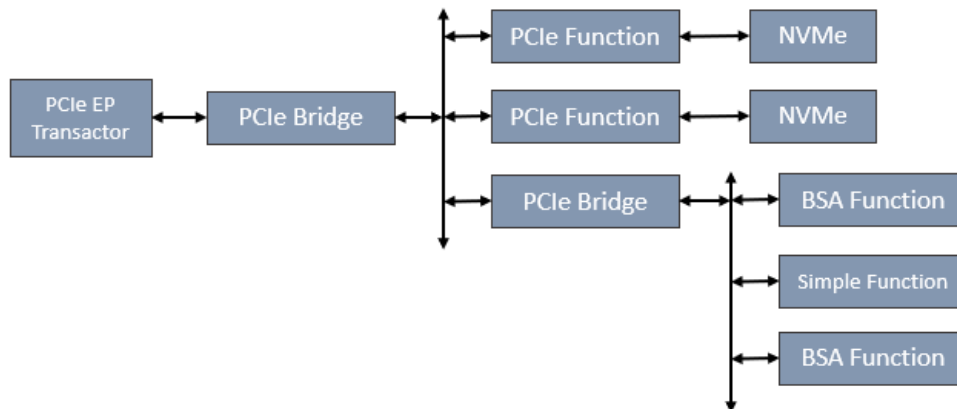
The simplest and most-frequently used PCIe topology is a single-function device. As an example, a NVMe device can be used as a target by defining the following topology:

```
<virtDev:PCIe>
  <virtDev:Port>
    <virtDev:Name>link0</virtDev:Name>
    <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
    ...
  </virtDev:Port>
  <virtDev:Device>
    <virtDev:Function>
      <virtDev:Name>NVMe</virtDev:Name>
      <virtDev:DebugLevel>1</virtDev:DebugLevel>
    </virtDev:Function>
  </virtDev:Device>
</virtDev:PCIe>
```

Arbitrary Topology

The following figure shows a complex topology created by user for a particular test scenario:

Figure 2 An example of a complex topology



The topology depicted above is created using the following XML configuration file:

```

<virtDev:PCIE>
  <virtDev:Port>
    <virtDev:Name>link0</virtDev:Name>
    <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
    ...
  </virtDev:Port>
  <virtDev:Bridge>
    <virtDev:Device>
      <virtDev:Function>
        <virtDev:Name>NVMe</virtDev:Name>
        <virtDev:DebugLevel>0</virtDev:DebugLevel>
      </virtDev:Function>
    </virtDev:Device>
    <virtDev:Device>
      <virtDev:Function>
        <virtDev:Name>NVMe</virtDev:Name>
        <virtDev:DebugLevel>0</virtDev:DebugLevel>
      </virtDev:Function>
    </virtDev:Device>
    <virtDev:Device>
      <virtDev:Function>
        <virtDev:Name>BSA</virtDev:Name>
      </virtDev:Function>
    </virtDev:Device>
    <virtDev:Bridge>
      <virtDev:Device>

```

```

        <virtDev:Function>
            <virtDev:Name>BSA</virtDev:Name>
            <virtDev:DebugLevel>0</virtDev:DebugLevel>
        </virtDev:Function>
        <virtDev:Function>
            <virtDev:Name>Simple</virtDev:Name>
        </virtDev:Function>
        <virtDev:Function>
            <virtDev:Name>BSA</virtDev:Name>
        </virtDev:Function>
    </virtDev:Device>
</virtDev:Bridge>
</virtDev:Bridge>
</virtDev:PCIe>
</virtDev:Config>

```

ARI Topology

The user-defined topology supports ARI devices. Note that ARI devices can only be implemented immediately downstream of the Root Complex, or a bridge. The functions are linked in the ARI Capability Next Function Number register in the order they are specified. For example, an eight-function ARI device can be as follows:

```

<virtDev:PCIe>
    <virtDev:Port>
        <virtDev:Name>link0</virtDev:Name>
        <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
        ...
    </virtDev:Port>
    <virtDev:Device>
        <virtDev:ARI>true</virtDev:ARI>
        <virtDev:Function>
            <virtDev:Name>Simple</virtDev:Name>
        </virtDev:Function>
        <virtDev:Function>
            <virtDev:Number>1</virtDev:Number>
            <virtDev:Name>Simple</virtDev:Name>
        </virtDev:Function>
        <virtDev:Function>
            <virtDev:Number>8</virtDev:Number>
            <virtDev:Name>Simple</virtDev:Name>
        </virtDev:Function>
        <virtDev:Function>
            <virtDev:Number>31</virtDev:Number>
            <virtDev:Name>Simple</virtDev:Name>
        </virtDev:Function>
        <virtDev:Function>
            <virtDev:Number>15</virtDev:Number>
            <virtDev:Name>Simple</virtDev:Name>
        </virtDev:Function>
        <virtDev:Function>

```

```
        <virtDev:Number>255</virtDev:Number>
        <virtDev:Name>Simple</virtDev:Name>
    </virtDev:Function>
    <virtDev:Function>
        <virtDev:Number>127</virtDev:Number>
        <virtDev:Name>Simple</virtDev:Name>
    </virtDev:Function>
    <virtDev:Function>
        <virtDev:Number>225</virtDev:Number>
        <virtDev:Name>Simple</virtDev:Name>
    </virtDev:Function>
</virtDev:Device>
</virtDev:PCIe>
</virtDev:Config>
```

Supported PCIe Functions

Following are supported PCIe functions:

- [Using the Test Functions](#)
- [User-Defined Functions](#)
- [Data Object Exchange \(DOE\)](#)
- [Integrity and Data Encryption \(IDE\)](#)
- [Configuring the Physical Layer](#)
- [Control Dynamic Lane Width](#)
- [Enabling PCIe Power Management](#)
- [Advanced Error Reporting \(AER\)](#)
- [Enabling PCIe or USB Transactor in ProxyEnv](#)
- [Support for 10-bit Tag](#)
- [Support for 14-bit Tag](#)

Using the Test Functions

VDM includes four predefined functions. These functions can be easily identified during enumeration and device probing using their respective vendor ID, device ID, and class code.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Device ID																16C3h												00h				
																																04h
Class Code																																08h

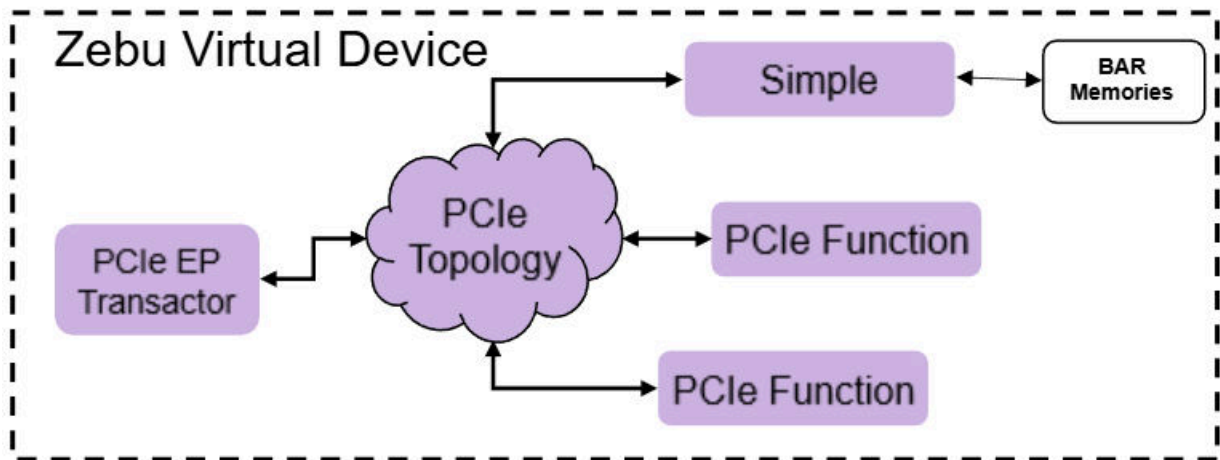
Field	Type	Description	Value
Vendor ID	RO	Synopsys Inc.	16C3h
Device ID	RO	Simple Function	5B58h
		NVMe Function	5B54h
		ATS/PRI Test Function	5B54h
		BSA Compliance Testing Functions	5B5Ah
		CXL Type1 Function	5B81h
		CXL Type2 Function	5B82h
		CXL Type3 Function	5B83h
Class Code	RO	Simple Function	005B5Ah
		NVMe Test Function	010802h
		ATS/PRI Test Function	005B54h
		BSA Test Function	005B5Ah
		CXL Function	050210h

Simple Function

The Simple test function offers minimal functionality to exercise basic PCIe capabilities, such as configuration accesses, downstream and upstream memory access. The Simple test function is instantiated by specifying the function name "Simple" in the <virtDev:Name> XML element.

```
<virtDev:Function>
  <virtDev:Name>Simple</virtDev:Name>
  <virtDev:DebugLevel>1</virtDev:DebugLevel>
</virtDev:Function>
```

Figure 3 VDM solution for a simple function



The function contains two memory regions. BAR[0] is a 32-bit address region with 32K bytes. BAR[1] is a 64-bit address region with 32K bytes. However, the physical memory behind them is hardcoded to 8192 bytes per region, which is sufficient 4096-byte accesses of any alignment. Accesses beyond this hard limit are simply wrapped around.

Upstream memory requests can be programmatically initiated by obtaining the function using the `SNPS::VSA::VDM::DeviceEnv::Env::getFunction()` method, and calling the `SNPS::VSA::VDM::PCIe::Function::memRd()`, `SNPS::VSA::VDM::PCIe::Function::memWr()`, `SNPS::VSA::VDM::PCIe::Function::DMWr()`, or `SNPS::VSA::VDM::PCIe::Function::atomicOp()` methods. Note that if a NULL data buffer is specified to the `SNPS::VSA::VDM::PCIe::Function::memRd()` method, the request is posted and the eventual response is ignored. If the `SNPS::VSA::VDM::PCIe::Function::memRd()`, `SNPS::VSA::VDM::PCIe::Function::DMWr()`, or `SNPS::VSA::VDM::PCIe::Function::atomicOp()` method returns the 'SNPS::VSA::VDM::PCIe::CRS' status, it indicates that the transactor is being overwhelmed with requests and the request should be retried at a later time.

If the function's C++ API is not accessible from the testcase (for example, it is running on a connected embedded CPU or VM), you can program a DMA engine in the function to generate upstream requests. The DMA engine is disabled by default. It is enabled by specifying the BAR index and offset within that BAR region where the control registers are located. The BAR index is specified in the least significant 3 bits of the value. Bits 63..3 of the value specify bits 63..3 of the offset. The offset is always DW-aligned and thus bits 2..0 are always zero. For example, to enable the DM engine at offset 0xF0000008 in BAR[1]:

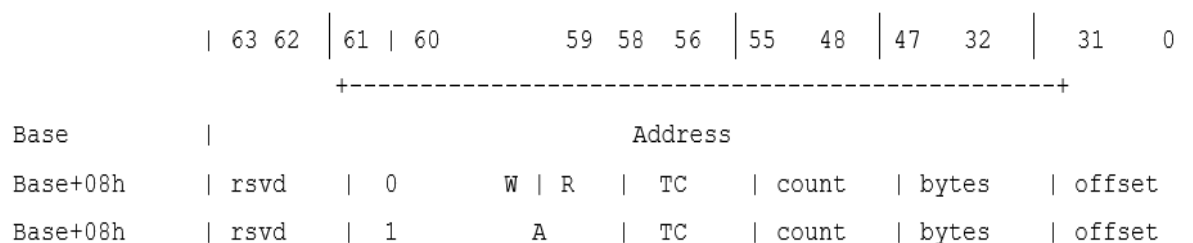
```
<virtDev:Function>
  <virtDev:Name>Simple</virtDev:Name>
```

```
<virtDev:UpReqBIR>0xF0000009</virtDev:UpReqBIR>
<virtDev:Function>
```

Note:

The offset may not be greater than 0x00007FF0 and the BAR index can only 0 or 1 as the BARs are pre-defined.

Upstream requests can be requested by sequentially writing two 64-bit values at the specified base address and the specified base address + 0x08. As soon as the second write is completed, the upstream request is performed. The request is specified using the following memory mapped fields, in little-endian order.



This requests a transfer between the specified upstream address and the specified offset in the same BAR region. The upstream request is repeated 'count' times, with the specified address and offset incremented by 'bytes' each time.

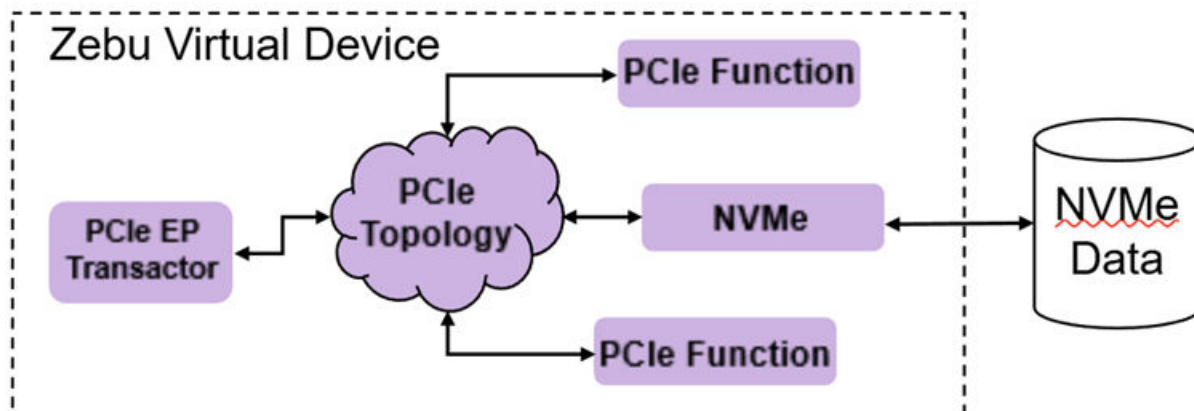
If a Read or Write operation is requested (bit 61 == 0), the write data (if W is set) is taken from the same BAR at the specified offset. The read data (if R is set) is stored in the same BAR at the specified offset. If both W and R are set, a write request is immediately followed by a read request at the same address.

If an atomic operation is requested (bit 61 == 1), the 'A' field specifies the atomic operation: A value of 0b00 specified an ADD operation, a value of 0b01 specifies a SWAP operation, and a value of 0b10 specifies a CAS operation. Th value 0b11 is reserved. The 'bytes' field specifies the size of the atomic operation and must be 4, 8, or 16 (CAS only). The operand value is taken from the same BAR at the specified offset and the result is stored at the same location. On repeated CAS operations, the offset is incremented by 2 x bytes.

NVMe Test Function

The NVMe test function is a subset of a NVMe 1.3 device as specified in the NVM Express Revision 1.3 specification document (https://nvmexpress.org/wp-content/uploads/NVM_Express_Revision_1.3.pdf). While not fully 1.3 compliant, it provides enough functionality to satisfy the needs of the standard Linux and Win10 NVMe drivers. The NVMe test function is instantiated by specifying the function name "NVMe" in the <virtDev:Name>XML element.

Figure 4 VDM solution for NVMe function



Message Signal Interrupt (MSI) and Message Signal Interrupt-Extended (MSIX) features are supported. You can enable MSI for multiple vectors by using `<virtDev:MSI>` XML element. In this XML config MSI is verified with 4 vectors.

MSIX is controlled by using `<virtDev:MSIX>` XML element. You can control it by using true / false values.

The default MSIX vector count is set to 16 when user passes MSIX as true.

```
<virtDev:Function>
  <virtDev:Name>NVMe</virtDev:Name>
  <virtDev:DebugLevel>0</virtDev:DebugLevel>
  <virtDev:NVMe>
    <virtDev:DebugLevel>1</virtDev:DebugLevel>
  </virtDev:NVMe>
  <virtDev:MSI>4</virtDev:MSI>
  <virtDev:MSIX>>false</virtDev:MSIX>
</virtDev:Function>
<virtDev:Function>
```

Several NVMe parameters can be configured in the optional `<virtDev:NVMe>` XML element if their default is not suitable.

NVMe Parameter	Description	Default
Debug Level	Debug verbosity of the NVMe layer functionality. It is separate from the PCIe Function's debug verbosity which reports on the PCIe layer. Must be an integer between 0 and 4.	0
MSI-X	Hide the MSI-X capability structure, forcing the host to use MSI. By default, both MSI and MSI-X capability structures are present.	True

NVMe Parameter	Description	Default
CAP.MQES	Maximum queue entry supported. Must be a value between 2 and 65536. Reflected as a zero-based value in MQES field in the CAP register. <i>Note:</i> A large value might cause timeouts at runtime if the NVMe function is not able to catch up with a deep submission queue.	4
NN	Number of namespaces. Specified by the number of <virtDev:Namespace>elements in the <virtDev:NVMe> element.	1
Namespace Configuration Parameters		
NSZE	Number of LBAs in the namespace.	0x10000
LBADS	Formatted LBA size, in bytes. Must be a power of 2. Minimum is 512. Note: Data is transferred one block at a time. Upstream TLPs are not larger than the block size plus metadata size.	512
MS	Formatted metadata size, per LBA, in bytes.	0
MSET	If true, metadata is transferred as part of an extended LBA. Note: Transfer of metadata separately from the LBA is not yet supported.	True
Image File	Name of the file where content that is not volatile is initialized from / saved to. The file is created if it does not exist. If the file already exists, the value of the NSZE, LBADS, MS, and MSET parameters of the namespace are forcibly set to those that existed in the namespace that created the file. If the file name ends with an .iso extension, it is assumed to have an ISO-9660 format and be read-only. The namespace configuration is forcibly set to match the configuration specified in the ISO file. In addition, the contents of the .iso file "initialize" the NVMe namespace. The ISO disk image file must exist. It is not possible to create an ISO file from scratch.	Nvme<instNum>n<nsNum>.nimg

Example XML configuration of all NVMe parameters in a single namespace NVMe device is as follows:

```
<virtDev:Function>
```

```
<virtDev:Name>NVMe</virtDev:Name>
<virtDev:NVMe>
  <virtDev:DebugLevel>1</virtDev:DebugLevel>
  <virtDev:MSIX>true</virtDev:MSIX>
  <virtDev:CAP>
    <virtDev:MQES>4</virtDev:MQES>
  </virtDev:CAP>
  <virtDev:Namespace>
    <virtDev:NSZE>0x80000000</virtDev:NSZE>
    <virtDev:LBADS>128</virtDev:LBADS>
    <virtDev:MS>0</virtDev:MS>
    <virtDev:MSET>true</virtDev:MSET>
    <virtDev:Filename>diskImage.iso</virtDev:Filename>
  </virtDev:Namespace>
</virtDev:NVMe>
<virtDev:Function>
```

Note:

To ease the transition from the Virtual NVMe Transactor to VDM, "legacy" Virtual NVMe Transactor XML configuration files are supported by the Device Environment object.

The NVMe function supports the following Admin commands:

- Identify Controller, Identify Namespace, Identify Namespace List
- Create Submission Queue (SQ) (contiguous queues only)
- Create Completion Queue (CQ) (contiguous queues only)
- Delete SQ, CQ
- Get Feature (SMART / Health information only)
- Set Feature (arbitration, number of queues, interrupt coalescing, asynchronous event configuration, and auto power transition only)
- Asynchronous Event Request (temperature threshold and auto-power transition only)
- Abort

The NVMe function supports the following NVMe commands: `Read` and `Write`.

Note:

Commands using scatter-gather lists, and fused commands are not supported.

Single Root I/O Virtualization (SR-IOV) Support

By default, the NVMe function is a pure Physical Function. It is possible to enable SR-IOV and have up to 255 Virtual NVMe functions. SR-IOV is enabled by adding a `<virtDev:SRIOV>` tag to the function's XML configuration. For example:

```
<virtDev:Function>
  <virtDev:Name>NVMe</virtDev:Name>
  <virtDev:DebugLevel>0</virtDev:DebugLevel>
  <virtDev:NVMe>
    <virtDev:SRIOV>
      <virtDev:TotalVFs>31</virtDev:TotalVFs>
      <virtDev:FirstOffset>0x0004</virtDev:FirstOffset>
      <virtDev:Stride>0x0004</virtDev:Stride>
    </virtDev:SRIOV>
  </virtDev:NVMe>
</virtDev:Function>
```

Note that each VF is completely independent of the Physical Function. It is not possible to access an NVMe namespace in a VF from the PF. SR-IOV support in VDM is currently limited to having all VFs on the same bus as the PF. For the maximum number of VFs (255), locate the PF in slot 00.0 and set FirstOffset and Stride to 0x0001.

Formatting and Mounting

When Linux starts, it enumerates and configures the NVMe function automatically. If successfully recognized, a device file named, `/dev/nvme<X>n<Y>` is created for every namespace "Y" in every NVMe function "X". Unless initialized from an existing file, namespaces are not initially formatted. To use the NVMe function as a file system, it is necessary to format and mount each namespace first.

To format a NVMe function namespace, use one of the following commands:

```
% fdisk /dev/nvme0n1
```

Or, for one single partition without user interaction:

```
% echo 'type=83' | sfdisk /dev/nvme0n1
```

Then, use the following command to create the file system on the newly formatted partition:

```
% mkfs /dev/nvme0n1p1
```

The process takes about 15 mins for a 2GB partition and creates an inode table, super-blocks, and file system information. The NSZE and LBADS configuration parameters define the total size of a NVMe namespace. The default is 32MB.

To mount the filesystem, use the following commands:

```
% mkdir /mnt/mydisk
% mount /dev/nvme0n1p1 /mnt/mydisk
```

The file system on the NVMe namespace is now available:

```
% ls /mnt/mydisk
% cp /a/large/file /mnt/mydisk
```

Note that writing to the NVMe file system might result in few or no WRITE transfers executed on the NVMe function. That is because the Linux OS buffers and caches the disk content for improved performance and only flushes updated blocks when necessary. To flush the disk I/O cache buffer and commit the updated blocks to the NVMe device, use one of the following commands:

```
% sync
```

or:

```
% hdparm -F /dev/nvme0n1
```

Enabling NVMe Analyzer for VDM

NVMe analyzer analyzes the PCIe TLPs (or transactions) and generates NVMe events and commands for the NVMe VDM. To enable the NVMe analyzer, add the following code to the XML configuration file:

```
<virtDev:Port>
  <virtDev:Name>link0</virtDev:Name>
  <virtDev:DebugLevel>2</virtDev:DebugLevel>
  <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
  <virtDev:Type>DPI</virtDev:Type>
  <virtDev:ProtocolAnalyzer>
    <virtDev:NVMe>
      <virtDev:IsEnabled>true</virtDev:IsEnabled>
      <virtDev:DebugLevel>1</virtDev:DebugLevel>
      <virtDev:ReportEveryMB>0</virtDev:ReportEveryMB>
    </virtDev:NVMe>
  </virtDev:ProtocolAnalyzer>
</virtDev:Port>

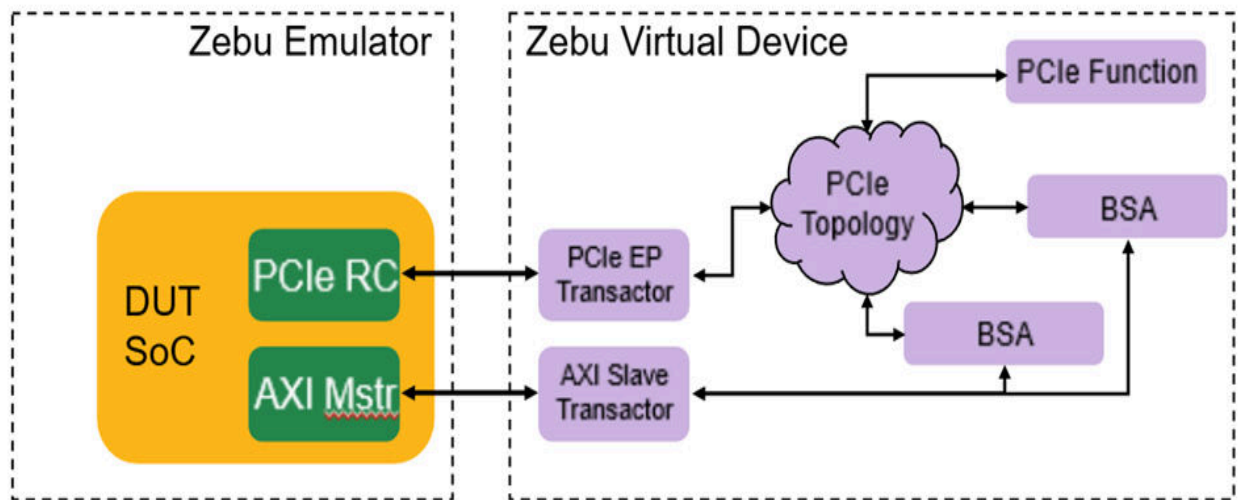
<virtDev:InactiveIfIdleForMicroSecs>5000</virtDev:InactiveIfIdleForMicroSecs>

<virtDev:CQDBTimeoutMicroSecs>3000</virtDev:CQDBTimeoutMicroSecs>
  </virtDev:NVMe>
</virtDev:ProtocolAnalyzer>
</virtDev:Port>
```

BSA Test Function

The BSA test functions provides the required functionality to execute the ARM BSA Compliance test suite. For more details, see the *BSA Compliance Testing Solution User Guide*.

Figure 5 VDM solution for BSA test function



ATS/PRI Test Function

The ATS/PRI Test Function is specifically designed to exercise the ATS/PRI protocol and verify the interaction between a function and a TA. All protocol interactions are triggered by user-implemented test software running on the host via memory-mapped registers. No actual functionality using translated addresses is provided by the function.

The user-implemented test software queues ATS and PRI requests in the function, then triggers them by ringing the "Execute Doorbell". All relevant responses received from the host are queued, in order, for subsequent retrieval by the test software to verify their correctness.

The ATS/PRI Test Function does keep track of successfully translated addresses and responds autonomously to invalidation requests. Otherwise, it does not interpret any other translation information, such as attributes.

For more information, see the following topics:

- [Configuration Space](#)
- [Memory-Mapped Registers](#)
- [Programming Model](#)

Configuration Space

0 shows the PCIe configuration space registers that are specific to the ATS/PRI Test Function. All other configuration registers and capabilities are as specified in the PCI Express Base Specification Revision 4.0 Version 1.0.

Figure 6 *ATS Extended Capability*

31

Next Cap Offset				1h	000Fh			0 Offset
E	AMA	Reserved	STU		Reserved	Bh	IQD	00h
								04h

Field	Type	Description	Initial Value
IQD	RO	Invalidate Queue Depth. Configurable at instantiation time.	0
STU	RW	Smallest Translation Unit	0
AMA	RW	ATS Memory Attributes configuration	0h
E	RW	ATS Enable. Must be set for the ATS functionality to operate.	0

For details on the various field semantics, refer to the PCIe specification.

Figure 7 *PRI Extended Capability*

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Next Cap Offset												1h		0013h																00h		
0h	Reserved				0h	Reserved				UPRGI	RF	Reserved										R	E	04h								
FFFFFFFh																		08h														
OPRA																		0Ch														

Field	Type	Description	Initial Value
E	RW	Enable. Must be set for PRI functionality to operate.	0
R	RW	Reset	0
RF	RW1C	Response Failure	0h
UPRGI	RW1C	Unexpected Page Group Request Index	0h
OPRA	RW	Outstanding Page Request Allocation. Ignored.	FFFFFFFFh

For details on the various field semantics, refer to the PCIe specification.

Memory-Mapped Registers

The following memory-mapped registers are used to configure and control the ATS/PRI Test Function. They are located at the specified offset in the BAR[0] region. All other offsets are reserved. All memory-mapped registers must be accessed using aligned DWORD (32 bits) accesses.

- [CSTS: Function Control and Status Register](#)
- [Pending Requests Status Register](#)
- [TX: ATS/PRI Requests Queue](#)
- [RX: ATS/PRI Response Queue](#)

CSTS: Function Control and Status Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
R																																00h
RXSIZ																COALESCE				CC		IVE		IRQ		GO						

Field	Type	Description	Initial Value
GO	W1	Send queued upstream memory read requests and PRI messages. Auto-clears when all have been sent.	0h
IRQ	RW	Send MSI when GO gets cleared. MSI must be configured and enabled.	0h
IVE	RW	Enable in validation. When cleared, Invalidation Requests are ignored.	1h
CC	RW	Value of CC field in ATS Invalidation Completion messages. Must not be changed while IVE is set.	0h
COALESCE	RW	Minimum number of ATS Invalidate completion to coalesce before sending “CC” consecutive Invalidation Completion messages. This is a zero-based value, where 00h indicates a minimum of 1 ITAG and a value of 1Fh indicates a minimum of 32 ITAG. Must not be changed while IVE is set	00h
RXSIZ	RO	Number of response DWs received from the host currently buffered. Saturates at FFh. When value is saturated, DWs are still received and buffered but not reported here.	00h
R	W1	Reset function. Auto-clears to 0 when reset is complete. Flushes all queues and buffers.	0h

Pending Requests Status Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
PRI																AT																08h

Reading Pending Request Status Register returns the number of pending address translation and page request groups that are still pending. Writing to this register causes the function to display a description of all pending requests.

Warning:

A warning is issued if there are pending requests when the function is destroyed.

Field	Type	Description	Initial Value
AT	RO	Number of pending address translation requests.	0000h
PRI	RO	Number of pending page request groups.	0000h

TX: ATS/PRI Requests Queue

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
TxDw																																10h

Writing to ATS/PRI Requests Queue register appends a DW to the ATS/PRI request queue. Reading has no effect and returns 0x00000000. Requests and messages are sent as fast as possible when CTST.GO is enabled. Writing to this register while GO is set has no effect.

DWs in the Tx Queue are interpreted in write order. The first DW defines how the subsequent DWs are going to be interpreted (if any). If more DWs are in the queue, the process repeats itself until no DWs remain. The first DW is interpreted as follows:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
NDWS								TAG								Reserved				Attr		AT		Reserved				REQ							
Field								Description																											
REQ								Request Type																											
								0h														32-bit Upstream Memory Read													
								1h														64-bit Upstream Memory Read													
								2h														32-bit Upstream Memory Write													

Field	Description	
	3h	64-bit Upstream Memory Write
	8h	PRI message. DW2 and DW3 of the message TLP must immediately follow and NDWS must be 02h.
AT	AT attribute bits. Ignored for PRI messages.	
Attr	Attribute bits	
TAG	Tag used to associate Address Translation responses and requests. The tag FFh is reserved and must not be used. Tags should not be reused until a translation request has been completed. Used to set the Tag field in PRI but not used to match PRI responses with requests.	
NDWS	Number of payload DWs or address translation requested. If Upstream Memory Write, the payload data DWs immediately follow the address DWs. If PRI message, must be set to 2.	

For 32-bit memory requests, the next DW is the 32-bit address. For 64-bit memory requests, the next two DWs are the 64-bit address with the most-significant DW first. Address and data payload DWs are in host-endian order, not network order.

The address bits are not masked nor interpreted in any way. If the lower-order bits are used to specify attributes (for example, the NW attribute in ATS requests), they are transmitted as-is.

RX: ATS/PRI Response Queue

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
RxDw																																10h

Reading from this register returns the next DW from the ATS/PRI response buffer queue. Writing has no effect. The value of CTST.RXSIZ specifies the number of DWs currently in the buffer that have not yet been read. Reading this register when the receive buffer is empty returns 00FFFFFFh.

DWs in the Rx Queue are to be interpreted in read order. The first DW defines how the subsequent DWs are to be interpreted (if any). If more DWs are in the queue, the process repeats itself until no DWs remain. The first DW is interpreted as follows:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDWS								TAG				Reserved		Attr		AT		Reserved			RSP										

Field	Description
RSP	Response Type

Field	Description	
	0h	Upstream Memory Read Completion. All payload DWs (if any) immediately follow.
	4h	ATS Invalidation message. Message body (containing the untranslated address range to invalidate) immediately follows. NDWS set to 02h.
	8h	PRI response message. DW2 of the message TLP (containing the response code) immediately follows. NDWS set to 01h.
	Fh	Receive buffer is empty. NDWS set to 00h.
Status	PCIe completion status of the received CplID TLP.	
TAG	Tag used to associate Address Translation responses and requests (whenever relevant and possible). A TAG value of FFh indicates a response not associated with a request. For ATS Invalidation, contains ltag field. For PRI Responses, contains Tag field.	
NDWS	Number of payload DWs. Pay load DWs immediately follow this DW.	

Programming Model

The following describes how the ATS/PRI tester function can be programmed to perform example tests:

Test for Address Translation Request

The following software sequence would send one ATS request for a 64-bit virtual address, then verify the translated response.

1. Write R==1 to CTST register to reset the tester function
2. Write 0x01AA0101 to TxDW register to start a new request for a single address translation
3. Write A[63:32] to TxDW as DW2
4. Write {A[31:12],0x000} to TxDW as DW3
5. Write Go == 1 to CTST register to send all queued DWs.
6. Wait until test is complete (interrupt, or GO == 0)
7. Check that RXSIZ == 3 in CSTS register
8. Read RxDW register and check that it is equal to 0x02AA0001 (Valid address translation response for one address)

9. Read RxDW register and check that it is equal to PhysAddr[63:32]
10. Read RxDW register and check that it is equal to PhysAddr[31:12] + Attributes

Test for Invalidation Request

The following software sequence would check a received invalidation request and issue invalidation request response message:

1. Write R == 1, COALESCE == 0x00, CC == n, and IVE = 1 to CTST register to reset and configure the tester function
2. Perform an Address Translation Request
3. Cause an invalidation request to be sent by the host
4. Read CTST register and check that RXSIZ == 3
5. Read RxDW register and check that it is equal to 0x02xx0004 (Invalidation request with ITag equal to "xx")
6. Read RxDW register and check that it contains the expected Untranslated Address[63:32]
7. Read RxDW register and check that it contains the expected Untranslated Address[32:12] and range.
8. Check that CC Invalidation Completion messages were received by the TA and the invalidation is complete

Test for Stressing the Translation Agent

The following software sequence would send a number of consecutive address translation requests, in an attempt to stress its translation and response capabilities (assuming here the ability to translate 255 addresses without intervening invalidations):

1. Write R == 1 to CTST register to reset the tester function
2. Repeat 255 number of times

Write 0x02nn0101 to TxDW register to start a new request for a single address translation

Write A[i][63:32] to TxDW as DW2

Write {A[i][31:12],0x000} to TxDW as DW3

1. Write Go == 1 to CTST register to send all queued address translation requests.
2. Wait until test is complete (interrupt, or GO == 0)
3. Verify accumulated Check that RXSIZ == 3*255 in CSTS register
4. Repeat 255 times

Read RxDW register and check that it is equal to 0x02mm0001 (Valid address translation response for one address. “mm” will reflect whatever tag was used in the request as indicated by “nn” in step 2)

Read RxDW register and check that it is equal to PhysAddr[mm][63:32]

Read RxDW register and check that it is equal to PhysAddr[mm][31:12] + Attributes

Test for Page Request

The following software sequence would send a Page Request message:

1. Write R == 1 to CTST register to reset the tester function
2. Write 0x02BB0008 to TxDW register to initiate a PR request
3. Write Page Address [63:32] to TxDW register
4. Write Page Address [31:12], PRGI, and LWR to TxDW register
5. Write Go == 1 to CTST to send the queue Page Request
6. Wait until interrupt or Go == 0
7. Read CTST register and check that RXSIZ == 4
8. Read RxDW register and check that it contains the expected DW0 of the PRG Response Message
9. Read RxDW register and check that it contains the expected DW1 of the PRG Response Message
10. Read RxDW register and check that it contains the expected DW2 of the PRG Response Message
11. Read RxDW register and check that it contains 0x00000000

User-Defined Functions

It is possible to implement your own functions with different behaviors. You can implement a user-defined PCIe function, dynamically load it through a shared library, and instantiate it in an arbitrary topology.

A user-defined function is implemented in an external C++ file as a class derived from the SNPS::VSA::VDM::PCIe::Function class and an instance factory function. It is compiled into a shared object, which is then loaded by the SNPS::VSA::VDM::DeviceEnv::Env at initialization time.

```
#include "PCIeTopology.hh"
```

```
class myFunction : public SNPS::VSA::VDM::PCIe::Function
{
public:
    myFunction()
        : SNPS::VSA::VDM::PCIe::Function("myFunction")
        {
            ...
        }
    ...
}

void*
factory() {
    return new myFunction();
}
```

The various virtual methods in the *SNPS::VSA::VDM::PCIe::Function* class can be overloaded to intercept configuration, downstream memory accesses, and downstream messages. Other methods can be called to issue upstream memory requests or send upstream messages. An additional protected APIs, such as *Function::configureBAR()* and *Function::setCfg()*, are available to configure and interact with the function's configuration space and other base functionality such as Data Object Exchange (DOE). Detailed documentation of the PCIe Function C++ API can be found in the Doxygen output included in the package.

The name of the function specified in the XML file must not be the name of one of the predefined function. Any unknown function name is assume to identify a user-defined function. The name of the factory function and the name of the shared library containing it must be specified in the PCIe Topology XML configuration file. The *SNPS::VSA::VDM::DeviceEnv::Env* instance dynamically opens the specified shared library and calls the specified factory function to create an instance of the user-defined function. If the user-defined function requires additional shared libraries to be loaded, they should be added as dependencies when the shared library is created.

```
<virtDev:Function>
  <virtDev:Name>myFunctionInst0</virtDev:Name>
  <virtDev:SharedLibrary>libMyFunc.so</virtDev:SharedLibrary>
  <virtDev:FactoryName>factory</virtDev:FactoryName>
</virtDev:Function>
```

Note the factory function is not specified with "C" linkage using the "extern "C"" attribute. The name of the factory function is suitably mangled into its C++ symbol name equivalent when calling it to create the DOE responder instance.

Data Object Exchange (DOE)

By default, PCIe functions do not publish a DOE capability. But if enabled, a DOE capability is published, and DOE requests are forwarded to a DOE responder and returned

Data Object responses are provided back to the host. The DOE responder, if used, must not be blocking, and return zero or more Data Object instances that form the response back to the host.

A DOE responder can be procedurally specified or dynamically loaded through a user-specified shared library and inserted in any PCIe Function. In the first case, the DOE responder is implemented in the verification environment and directly registered with the function. In the second case, the DOE responder is implemented in an external C++ file, compiled into a shared object, which is then factory-instantiated by the PCIe Function at initialization time. The name of the DOE responder factory and the name of the shared library can be specified via the PCIeEnv topology XML configuration or explicitly at runtime when enabling DOE.

```
#include "vdm/PCIe/DOE.hh"

using DOE = SNPS::PCIe::DOE;

class myDOEResponder : public DOE::Responder {
    myDOEResponder()
        : DOE::Responder("myDOE")
    {
        supportedTypes(0x16C3, {0x00, 0x01, 0x05});
    }

    bool handleReq(uint16_t reqId,
                   const DOE::DataObject &req,
                   std::vector<DOE::DataObject> &rsps) override
    {
        if (DOE::Responder::handleReq(reqId, req, rsps)) return true;
        ...
    }
};

void* createDOEResponder() { return new myDOEResponder; }
```

Explicitly registered:

```
SNPS::VSA::VDM::DeviceEnv::Env myEnv(...);
auto fn = myEnv.getFunction(1, 0, 0);
fn->enableDOE(new myDOEResponder());
```

Dynamically loaded:

```
SNPS::VSA::VDM::DeviceEnv::Env myEnv(...);
auto fn = myEnv.getFunction(1, 0, 0);
fn->enableDOE("libDOE.so", "createDOEResponder");
```

A DOE responder can also be dynamically loaded and attached to a specific PCIe function by the SNPS::VSA::VDM::DeviceEnv::Env during initialization, by specifying the name

of the shared library and the name of the factory function in the XML specification for the function:

```
<virtDev:Function>
...
<virtDev:DOE>
  <virtDev:SharedLibrary>libmyDOE.so</virtDev:SharedLibrary>
  <virtDev:FactoryName>myResponderFactory</virtDev:FactoryName>
</virtDev:DOE>
</virtDev:Function>
```

Note the factory function is not specified with "C" linkage using the "extern "C"" attribute. The name of the factory function is suitably mangled into its C++ symbol name equivalent when calling it to create the DOE responder instance.

Multiple DOE instances can be enabled in the same PCIe Function by calling the `enableDOE()` method multiple times or by specifying multiple `virtDev:DOE` elements in the XML configuration file. Each DOE instance publishes its own instance of the DOE Capability in the Function's extended PCIe capability space. Each DOE instance should be configured with different DOE responder instances.

If the DOE responder instance is discovered to support the Secure CMA protocol (vendor ID 0x0001 and type 0x02), the IDE Extended Capability Registers are automatically published by the function.

Detailed documentation of the DOE base responder C++ API can be found in the Doxygen output included in the package.

Integrity and Data Encryption (IDE)

Note:

Using IDE requires a special configuration of the PCIe transactor. For details on instantiating the secure engine with the transactor, refer to *ZeBu® PCIe Transactor User Guide*.

By default, PCIe functions do not publish an IDE capability but if IDE is enabled, an IDE capability is automatically published. Further, IDE is only supported between the host and the root of the PCIe topology. Therefore, IDE can only be enabled in function 0 in a device that is directly connected to the host's downstream port.

IDE can be enabled for a function procedurally by calling the `SNPS::VSA::VDM::PCIe::Function::enableIDE()` method. In that case, key management requests should also be performed procedurally via the `SNPS::VSA::VDM::PCIe::Function::handleKeyManagement()` method.

IDE can be enabled for a function via XML by specifying the `<virtDev:IDE>` XML element in the XML configuration for that function. When enabling IDE via XML, it is also possible

to specify predefined transport layers for the IDE_KM requests so they can be performed by the host via CfgRd and CfgWr transactions rather than procedurally in C++. Two predefined KM transport layers are available: DOE and SPDm-over-DOE. The following XML excerpt shows two functions, each configured to use different KM transport layers. Note that a function can be configured with both KM transport layers. It would be up to the software running on the host to pick one to use.

```
<virtDev:Function>
...
<virtDev:IDE>
  <virtDev:KM>
    <virtDev:Transport>DOE</virtDev:Transport>
  </virtDev:KM>
</virtDev:IDE>
<virtDev:Function>
<virtDev:Function>
...
<virtDev:IDE>
  <virtDev:KM>
    <virtDev:SPDM>
      <virtDev:Transport>DOE</virtDev:Transport>
    ...
  </virtDev:SPDM>
</virtDev:KM>
</virtDev:IDE>
</virtDev:Function>
```

Pre-Shared Keys

When verifying a secure host, it is possible to avoid having to exchange keys between the host and the device at runtime using the IDE_KM protocol. Pre-shared keys can be configured in a function to secure a stream. Pre-shared keys are specified for each IDE stream, as sets of six keys, one for each direction for each sub stream.

```
<virtDev:PreSharedKeys>
  <virtDev:AutoSecure>true</virtDev:AutoSecure>
  <virtDev:StreamId>1</virtDev:StreamId>
  <virtDev:TrafficClass>7</virtDev:TrafficClass>
  <vHost:KeySet>
    <vHost:PR>

      <vHost:TxKey>000102030405060708090a0b0c0d0e0f202122232425262728292a2b2c2
d2e2f</vHost:TxKey>
      <vHost:TxIV>7071727374757677</vHost:TxIV>

      <vHost:RxKey>a0a1a2a3a4a5a6a7a8a9aaabacadaeaf808182838485868788898a8b8c8
d8e8f</vHost:RxKey>
      <vHost:RxIV>f0f1f2f3f4f5f6f7</vHost:RxIV>
    </vHost:PR>
```

```
<vHost:NPR>

<vHost:TxKey>101112131415161718191a1b1c1d1e1f303132333435363738393a3b3c3
d3e3f</vHost:TxKey>
  <vHost:TxIV>6061626364656667</vHost:TxIV>

<vHost:RxKey>b0b1b2b3b4b5b6b7b8b9babbbcbdbef909192939495969798999a9b9c9
d9e9f</vHost:RxKey>
  <vHost:RxIV>e0e1e2e3e4e5e6e7</vHost:RxIV>
</vHost:NPR>
<vHost:CPL>

<vHost:TxKey>202122232425262728292a2b2c2d2e2f404142434445464748494a4b4c4
d4e4f</vHost:TxKey>
  <vHost:TxIV>5051525354555657</vHost:TxIV>

<vHost:RxKey>c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0a1a2a3a4a5a6a7a8a9aaabaca
daef</vHost:RxKey>
  <vHost:RxIV>d0d1d2d3d4d5d6d7</vHost:RxIV>
</vHost:CPL>
</vHost:KeySet>
<vHost:KeySet>
  <vHost:PR>

<vHost:TxKey>C38F368C6BDC3E949D299C218C22717A1C5E0961D836AB84DB66AC8351E
D394C</vHost:TxKey>
  <vHost:TxIV>AD898F367CF5E0F8</vHost:TxIV>

<vHost:RxKey>A3DC4EC10B19B0DC929BD646821BC0D454788196C52F41112B7A3247AEC
AA214</vHost:RxKey>
  <vHost:RxIV>1BE2252859C05C99</vHost:RxIV>
</vHost:PR>
<vHost:NPR>

<vHost:TxKey>CE8AAA349CCE1EE283D9731BECCE21341BD5A4C82E04A3A39A7082B82FF
E9A80</vHost:TxKey>
  <vHost:TxIV>F387637794B8A82B</vHost:TxIV>

<vHost:RxKey>B34315EE5F23189D435414CB73088DEA12329B6484D998CF735730C48D2
C7FAE</vHost:RxKey>
  <vHost:RxIV>2441D94BB867125E</vHost:RxIV>
</vHost:NPR>
<vHost:CPL>

<vHost:TxKey>7E9D27BCF9E13052E1802021B6F827DA213F9CF62264592F781DA33E899
718D7</vHost:TxKey>
  <vHost:TxIV>2B2B7084B8F41834</vHost:TxIV>

<vHost:RxKey>39564B10C0C9714575D2E22A64A37E9132C430880D7B5DF64F61B1EB9DE
89C55</vHost:RxKey>
  <vHost:RxIV>2DB3A75374C1C151</vHost:RxIV>
</vHost:CPL>
</vHost:KeySet>
```

```

    <vHost:KeySet>
      <vHost:PR>

        <vHost:TxKey>7E9D27BCF9E13052E1802021B6F827DA213F9CF62264592F781DA33E899
        718D7</vHost:TxKey>
          <vHost:TxIV>D72B5013F470A211</vHost:TxIV>

        <vHost:RxKey>F40F582E82AA0C7005206A266654CC20E7E2EF4714096662183B9B4E181
        86781</vHost:RxKey>
          <vHost:RxIV>CBE5D1A45C4DDEEF</vHost:RxIV>
        </vHost:PR>
      <vHost:NPR>

        <vHost:TxKey>AF5DF44972BA10FE15A98FBEBBF10AA8D9F16F690C1C17B28B6447B9BD5
        64013</vHost:TxKey>
          <vHost:TxIV>83D3E166A1B15DAD</vHost:TxIV>

        <vHost:RxKey>89C7A012C62F832AFB1B920D3135512B0FEF1AF8F66270CFEEBC68F3FF5
        CD51F</vHost:RxKey>
          <vHost:RxIV>2E61C5682EA3D960</vHost:RxIV>
        </vHost:NPR>
      <vHost:CPL>

        <vHost:TxKey>13351C49960F6FD42674A06E5DC3219D43B4211914A3D48DADDAB52E69A
        2E3CB</vHost:TxKey>
          <vHost:TxIV>3EB8C41518914E9B</vHost:TxIV>

        <vHost:RxKey>D4DF8A75D98C354A89050A63F3C990F6EBCA16533442D52C1C215F1677D
        51349</vHost:RxKey>
          <vHost:RxIV>56134B4A9C487D4C</vHost:RxIV>
        </vHost:CPL>
      </vHost:KeySet>
    </virtDev:PreSharedKeys>

```

Before an IDE stream can be used, it must be secured. The securing process normally involves the host sending keys and making the stream secure by using the IDE_KM protocol. But with pre-shared keys, a different process must be used.

If a pre-shared key stream is specified as *auto secured*, the keys and stream are automatically configured in the PCIe link and the stream enabled and secure in next available link or selective stream register block in the IDE Extended Capability whenever the function Memory Space is enabled. All subsequent non-configuration TLPs in the specified traffic class are immediately sent and expected to be received as encrypted TLPs.

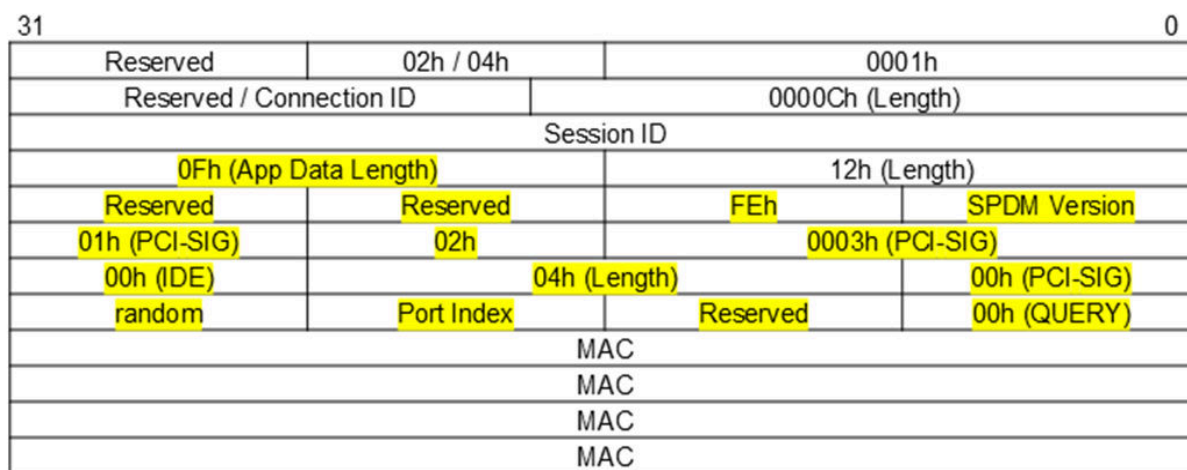
If a pre-shared key stream is not specified as *auto-secured*, the configuration of the keys and enabling and securing of the stream is triggered by writing the value 0x16C3ss02 to the IDE Capability register in the IDE Extended Capability, where “ss” is the logical stream id specified in the XML configuration file. The next available link or selective stream register block will be used. All subsequent non-configuration TLPs in the specified traffic class are immediately sent and expected to be received as encrypted TLPs.

If the stream is already secure, the request to secure the stream is ignored.

IDE_KM Over SPDM

Security Protocol Data Management (SPDM) is a protocol for securely exchanging secret key information over a clear channel, similar to establishing a *https://* session over the web. After the secure channel is established, it can be used to exchange IDE_KM messages.

Secure IDE_KM-over-SPDM messages are DOE-encapsulated messages using Vendor ID 0x0001 (PCI-SIG) and type 0x02 or 0x04. For example, an IDE_KM QUERY message would be encapsulated as follows, with the highlighted fields encrypted.



The establishment of a secure SPDM session is a lengthy multi-step process that ultimately results in the creation of a secure channel for exchanging IDE_KM messages. If the verification objective is not to verify SPDM but IDE, that is a significant overhead that does not contribute to the end goal. A function can be configured with pre-established secure SPDM sessions. If the host can similarly have secure sessions provisioned in advance, the entire SPDM stack can be short-circuited.

```
<virtDev:Function>
...
<virtDev:IDE>
  <virtDev:KM>
    <virtDev:SPDM>
      <virtDev:Transport>DOE</virtDev:Transport>
      <virtDev:SecureSession>
        <virtDev:SessionId>0xABCD1234</virtDev:SessionId>
        <virtDev:Algorithm>AES-128</virtDev:Algorithm>
        <virtDev:ReqKey>...</virtDev:ReqKey>
        <virtDev:ReqIV>...</virtDev:ReqIV>
        <virtDev:RspKey>...</virtDev:RspKey>
      </virtDev:SecureSession>
    </virtDev:SPDM>
  </virtDev:KM>
</virtDev:IDE>
</virtDev:Function>
```

```

        <virtDev:RspIV>...</virtDev:RspIV>
    </virtDev:SecureSession>
</virtDev:SPDM>
</virtDev:KM>
</virtDev:IDE>
...
</virtDev:Function>

```

Secure sessions can be configured to use the ‘AES-128’ or ‘AES-256’ ciphers. Request (host-to-function) and response (function-to-host) keys and Initial Values must be specified as hex literals, specifying 16 bytes and 32 bytes keys and 12 bytes Initial Values respectively. Sequence counts are initialized to 0 when the function is created and reset.

An IDE_KM-over-SPDM message can also be handled unsecurely with every field transferred in the clear. This assumes a similar capability exists in the host under a special test mode or device driver.

An unsecure IDE_KM message can be handled as a “secure” DOE request using types 0x02 or 0x04, using a SPDM session configured to be unencrypted, with the following layout. This layout is expected by the function if the SPDM session is configured with the “None” cipher. How the host is configured with an unencrypted “secure” session is host-specific.

31				0
Reserved		02h / 04h		0001h
Reserved / Connection ID			00008h (Length)	
Session ID				
0Fh (App Data Length)			11h (Length)	
Reserved		Reserved		FEh SPDM Version
01h (PCI-SIG)		02h		0003h (PCI-SIG)
00h (IDE)		04h (Length)		00h (PCI-SIG)
		Port Index		Reserved 00h (QUERY)

```

<virtDev:Function>
...
<virtDev:IDE>
    <virtDev:KM>
        <virtDev:SPDM>
            <virtDev:Transport>DOE</virtDev:Transport>
            <virtDev:SecureSession>
                <virtDev:SessionId>0xABCD1234</virtDev:SessionId>
                <virtDev:Algorithm>None</virtDev:Algorithm>
            </virtDev:SecureSession>
        </virtDev:SPDM>
    </virtDev:KM>
</virtDev:IDE>
</virtDev:Function>

```

An unsecure IDE_KM message can also be handled as an unsecure DOE request using types 0x01 or 0x03 with the following layout. This layout is handled by the function if no SPDm sessions are defined. How the host is configured to issue IDE_KM via an unsecure SPDm channel is host-specific.

31			0
Reserved	01h / 03h	0001h	
Reserved / Connection ID		00006h (Length)	
Reserved	Reserved	FEh	SPDM Version
01h (PCI-SIG)	02h	0003h (PCI-SIG)	
00h (IDE)	04h (Length)		00h (PCI-SIG)
	Port Index	Reserved	00h (QUERY)

```

<virtDev:Function>
...
  <virtDev:IDE>
    <virtDev:KM>
      <virtDev:SPDM>
        <virtDev:Transport>DOE</virtDev:Transport>
      </virtDev:SPDM>
    </virtDev:KM>
  </virtDev:IDE>
</virtDev:Function>

```

IDE_KM Over DOE

The establishment of a secure SPDm session is a lengthy multi-step process that ultimately results in the creation of a secure channel for exchanging IDE_KM messages. If the verification objective is not to verify SPDm but IDE, that is a significant overhead that does not contribute to the end goal. Synopsys provides an unsecure channel for exchanging IDE_KM messages between the RC and a function, short-circuiting the entire SPDm stack. Note that you need to limit the use of this channel only to verification; do not use it in the end product.

The IDE_KM messages are simply encapsulated in a DOE message with vendor ID 0x16C3 is a 16-bit field and data object type 0x01. If a function publishes a DOE capability that when probed via the Discovery process, advertises support for this protocol, it is assumed it supports IDE_KM-over-DOE. 0x00003 (length) is a 20-bit field.

Consider the following IDE_KM QUERY encapsulated in a Synopsys IDE_KM-over-DOE object as an example.

31			0
Reserved	0x01	0x16C3	
Reserved		0x00003 (Length)	
Port Index	Reserved	00h (Query)	00h (IDE)

Key Swapping

The endpoint automatically swaps Rx keys whenever the K bit is toggled by the host. To trigger a Tx key swap on a IDE stream, write the value 0x16C303nn to the function's Class Code register where "nn" is the IDE stream ID. Keys for all three substreams are swapped.

If pre-shared keys are used, a new set of keys is programmed in the unused K slot as soon as keys are swapped. Pre-shared keys are used in sequential order. When the last pre-shared key has been used, they are reused in the same order, starting with the first key. If IDE_KM is used, the host should program new keys in the unused K slot before the next key swap happens.

TEE Device Interface Security Protocol (TDISP)

Note:

Due to a limitation in the transactor, it is not currently possible to determine if a request had the T bit set nor whether it was received over a secure IDE stream.

Note:

Tracking of modifications to the IDE stream configuration registers is not currently supported.

TDISP support is an optional feature on top of IDE. It is disabled by default.

TDISP can be enabled for a function via XML by specifying the `<virtDev:TDISP>` XML element in the XML configuration for that function.

```
<virtDev:Function>
  ...
  <virtDev:IDE>
    ...
    <virtDev:TDISP>
      <virtDev:Enable>true</virtDev:Enable>
    ...
  </virtDev:TDISP>
</virtDev:IDE>
</virtDev:Function>
```

Currently, only the following requests and responses are supported:

- GET_TDISP_VERSION / TDISP_VERSION
- GET_TDISP_CAPABILITIES / TDISP_CAPABILITIES
- LOCK_INTERFACE_REQUEST / LOCK_INTERFACE_RESPONSE
- GET_DEVICE_INTERFACE_REPORT / DEVICE_INTERFACE
- GET_DEVICE_INTERFACE_STATE / DEVICE_INTERFACE_STATE

- START_INTERFACE_REQUEST / START_INTERFACE_RESPONSE
- STOP_INTERFACE_REQUEST / STOP-INTERFACE_RESPONSE
- TDISP_ERROR

By default, TDISP messages that were not received over a secure channel are ignored (as per section 11.2.2 of the protocol specification). For development and testing purposes, it is possible to enable unsecure transport layers for TDISP messages so they can be exchanged without encryption, making debug and analysis easier. Three predefined unsecure transport layers are available: raw TDISP messages over DOE, unsecure SPDM messages over DOE, and unencrypted secure SPDM messages over DOE. Raw TDISP messages over DOE are enabled only if the raw IDE-KM over DOE transport layer is also enabled.

```
<virtDev:TDISP>  
  <virtDev:Enable>true</virtDev:Enable>  
  <virtDev:Unsecure>true</virtDev:Unsecure>  
</virtDev:TDISP>
```

TDISP can also be enabled procedurally when calling the `SNPS::VSA::VDM::PCIE::Function::enableIDE()` method. In that case, TDISP requests should also be performed procedurally via the `SNPS::VSA::VDM::PCIE::Function::handleTDISP()` method.

Unsecure TDISP Messages

It is possible to exchange TDISP messages unsecurely, in violation of section 11.2.2 of the TDISP specification. This must only be done during the development and testing phases of the device and must not remain in the final commercial device. How these unsecure transport mechanisms are implemented in the host is outside the scope of this document.

Raw TDISP Messages Over DOE

This is the simplest TDISP message format. It can only be enabled if the same raw IDE-KM message transport layer is enabled. It does not require that raw IDE-KM messages be used to establish a secure link prior to exchanging TDISP messages, nor is a secure link required to exchange TDISP messages. TDISP messages are simply encapsulated in a DOE data object using vendor ID 0x16C3 (SNPS) and Protocol ID 0x02.

For example, a GET_TDISP_VERSION message would be sent using the following DOE data object:

31			0
Reserved	02h	16C3h	
Reserved		00006h (Length)	
Reserved		81h	10h
	Function ID		
	Reserved		

Unsecure SPDM Messages

TDISP messages are exchanged using unsecure SPDM messages, vendor 0x0001 (PCI-SIG) and type 0x01 or 0x03 (unsecure). This transport layer does not require that the link be secure nor that a SPDM session be established.

31			0
Reserved	01h / 03h	0001h	
Reserved/ Connection ID		00009h (Length)	
Reserved	Reserved	FEh	SPDM Version
01h (PCI-SIG)	02h	0003h (PCI-SIG)	
01h (TDISP)	04h (Length)		00h (PCI-SIG)
Reserved		81h	10h
	Function ID		
	Reserved		

Clear-Text Secure SPDM Messages

TDISP messages are exchanged using secure but unencrypted SPDM messages, vendor 0x0001 (PCI-SIG) and type 0x02 or 0x04 (secure). This transport layer does not require that the link be secure, but it requires that a clear-text SPDM session be established.

31			0
Reserved	02h / 04h	0001h	
Reserved/ Connection ID		000011h (Length)	
	Session ID		
1Ch (App Data Length)		1Eh (Length)	
Reserved	Reserved	FEh	SPDM Version
01h (PCI-SIG)	02h	0003h (PCI-SIG)	
01h (TDISP)	04h (Length)		00h (PCI-SIG)
Reserved		81h	10h
	Function ID		
	Reserved		

Configuring the Physical Layer

A few physical layer parameters are configurable when specifying a PCIe interface. For more information, see the following sections:

- [Maximum Payload Size](#)
- [Maximum Link Speed](#)
- [Cycle Accurate Mode](#)

Maximum Payload Size

The maximum payload size (MPS) advertised by the endpoint transactor can be configured using the optional `<virtDev:MaxPayloadBytes>` XML element. The value specified must be one of 128, 256, 512, 1024, 2048, or 4096. The actual MPS value used over a PCIe link eventually depends on the negotiated MPS value between the root complex and the transactor.

```
<virtDev:PCIe>
  <virtDev:Port>
    <virtDev:Name>link0</virtDev:Name>
    <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
    <virtDev:MaxPayloadSize>512</virtDev:MaxPayloadSize>
    ...
  </virtDev:Port>
  ...
</virtDev:PCIe>
```

By default, the transactor advertises being capable of a maximum payload size of 4096 bytes.

Maximum Link Speed

The maximum link speed advertised by the root device can be configured using the optional `<virtDev:MaxLinkSpeed>` XML element. The value specified must be 0, 1, 2, 3, 4, or 5, referring to the PCIe "Generation" number, that is, "5" means "Gen5 speed". If "0" is specified, the maximum speed supported by the transactor is used by default. The actual link speed used over a PCIe link eventually depends on the negotiated speed between the root complex and the transactor.

```
<virtDev:PCIe>
  <virtDev:Port>
    <virtDev:Name>link0</virtDev:Name>
    <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
    <virtDev:MaxLinkSpeed>4</virtDev:MaxLinkSpeed>
    ...
  </virtDev:Port>
  ...
</virtDev:PCIe>
```

By default, the transactor advertises being capable of its highest speed which might be proper.

Cycle Accurate Mode

By default, for best runtime performance, the PCIe transactor is free running, allowing other elements of the emulation to execute as quickly as possible. However, if it is necessary to measure cycle-accurate performance metrics, the PCIe transactor can be put in cycle-accurate mode using the `<virtDev:CycleAccurate>` XML element.

When in cycle-accurate mode, the PCIe transactor runs only when something is transmitted on the PCIe interface, or while an outstanding upstream transaction remains to be completed. This reduces run-timer performance of the emulation run but allows the capture of accurate performance metrics.

```
<virtDev:PCIe>
  <virtDev:Port>
    <virtDev:Name>link0</virtDev:Name>
    <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
    <virtDev:CycleAccurate>true</virtDev:CycleAccurate>
    ...
  </virtDev:Port>
  ...
</virtDev:PCIe>
```

Additionally, in the cycle accurate mode, link training starts immediately after a transactor (`initBFM()`) is instantiated. Based on the load on CPU, this API call may add indeterministic delay to the start time of the link training.

To get the deterministic link training `<virtDev:ProtocolCyclesToStartLinkTraining>` parameter is used to start the link training after specific PCIe clock cycle delays. This parameter is added at the `<virtDev:Port>` level for each link to start the link training after the specified cycles.

In case of multiple links, `initBFM()` is called sequentially. To ensure that the start time of the link training of all links is at the same time, first check the setup when the link training starts for the first link instance. Then, multiply this number with the number of links and set this value for `<virtDev:ProtocolCyclesToStartLinkTraining>`. The default is 0.

```
<virtDev:PCIe>
  <virtDev:Port>
    <virtDev:Name>link0</virtDev:Name>
    <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
    <virtDev:CycleAccurate>true</virtDev:CycleAccurate>

    <virtDev:ProtocolCyclesToStartLinkTraining>65536</virtDev:ProtocolCycles
    ToStartLinkTraining>
    ...
  </virtDev:Port>
```

```
...  
<virtDev:PCIe>
```

You can run the examples in the *Examples/PCIe/MultiLink* and *Examples/CXL/MultiLink* folders for PCIe and CXL, respectively.

Control Dynamic Lane Width

VDM supports validating change in the lane width of a PCIe transactor. It can be initiated by RC or EP.

To validate the lane width, use the following API:

```
bool changeLink(uint32_t width, bool iIOVcInitiated = false);
```

To initiate link width change, this API is initiated with valid link width. Valid link width values are 0, 1, 2, 4, 8 and 16. width = 0 implies maximum width.

`isRcInitiated` is an optional parameter that is only required for IP_ROOT versions earlier than S-2023.03-SP1. For S-2023.03-SP1 or later versions, the optional parameter can be used as a workaround for passing the width information change from the RC to the EP side because this feature is built-in.

Enabling PCIe Power Management

To enable PCIe power management using *Programmed Power Management (PPM)* and *Active-State Power Management (ASPM)*, use one or more of the following XML options to enable the presence of specific power management states:

```
<virtDev:Function>  
  ....  
  ....  
  <virtDev:LTR>  
    <virtDev:IsPresent>>false</virtDev:IsPresent>  
  </virtDev:LTR>  
  <virtDev:PM>  
  
  <virtDev:ClockPowerManagement>>true</virtDev:ClockPowerManagement>  
    <virtDev:States>  
      <virtDev:Dn>D0</virtDev:Dn>  
      <virtDev:Dn>D1</virtDev:Dn>  
      <virtDev:Dn>D2</virtDev:Dn>  
      <virtDev:Dn>D3</virtDev:Dn>  
      <virtDev:L1PM>L1.1</virtDev:L1PM>  
      <virtDev:L1PM>L1.2</virtDev:L1PM>  
      <virtDev:ASPM>L0s</virtDev:ASPM>  
      <virtDev:ASPM>L1</virtDev:ASPM>  
      <virtDev:ASPM>L1.1</virtDev:ASPM>
```

```
        <virtDev:ASPM>L1.2</virtDev:ASPM>
    </virtDev:States>
</virtDev:PM>
....
....
</virtDev:Function>
```

Every PCIe function has its independent power management control from XML.

Currently, power management capabilities are only supported in single-function devices.

D-State Transitions

A function can be transitioned to a different D-state by writing to the PowerState filed in the PMCSR register. All D-state transitions trigger a notification to any functor registered via the `Function::setUpdateNotifier()` method. As per the PCIe specification, it is the responsibility of the software to transition a function to a lower-power state after all pending transactions have completed.

When in D1 (if enabled), D2 (if enabled), or D3 state, downstream and upstream memory requests are refused with an UNSUPPORTED response but local access using the Function class APIs continue to work. The `Function::getDnState()` method can be used to determine the current D-state of a function.

A user-defined function may signal a “wake-up” event to the function to bring it into the D0 state and signal the interface to go to L0 by writing “00” to the PowerState field in the PMCSR register using the `Function::cfgWr()` method.

L-State Transitions

A function requests the interface to transition to PM-L1 state whenever it transitions to D1, or D2 state. After it is in the PM-L1 state, it may transition to a substate or to L2 state if the upstream port successfully negotiates for it. A function requests the interface to eventually transition to PM-L3 state whenever it transitions to D3 state.

The interface may transition to ASPM-L0s, ASPM-L1 (then eventually to ASPM L1.1 or ASPM L1.2) if so initiated by the upstream port. To initiate device-initiated transition to ASPM L1 state, write the value 0x16C30501 to the Class Code register of the function. Whether the interface eventually transitions to ASPM L1.1 or ASPM L1.2 is entirely dependent on the upstream port.

.

Advanced Error Reporting (AER)

By default, PCIe functions do not publish an AER capability structure. Each function instance can be configured to advertise such a capability. The presence of the AER and

capabilities of the AER capability structure are enabled by calling the following Function method:

```
bool enableAER(uint8_t numHeaderRecords = 1);
```

If the number of Header Records specified is greater than one, the Multiple Header Recording Capability is advertised. Enabling AER in a function that was not already publishing the capability does not imply that it reports errors through it. It is most likely to be only usable to inject error reports to verify the error handling capabilities of the device driver. AER injections can be performed using the following Function method:

```
bool Function::injectAER(bool correctable, uint8_t errIndex, uint8_t numDWs = 0, uint32_t* hdrDWs = NULL);
```

Injected AER errors are reflected in the Device Status register and each cause an error message to be sent to the Root Complex (if not masked). Multiple errors can be injected. If the number of unmasked errors with header records exceeds the number of Header Records available in the function, the Header Log Overflow error is indicated as well as the user-specified error but the header is not recorded. Identical errors are not coalesced and is signaled immediately after an earlier identical error is cleared. The First Error Pointer is updated based on the order of the error injections.

Advanced Error injections can also be performed in-band by writing the value 0x16C306nn to a function's Class Code register where "nn" is 0xFF to first enable AER if not already done, then the value for "correctable" and "errIndex". Bit 7 is used as the value for "correctable" and bits 4.0 are used for "errIndex". Bits 5 and 6 are reserved. For example, to inject a (correctable) "Bad DLLP" error, write 0x16C306807

Enabling PCIe or USB Transactor in ProxyEnv

VDM supports enabling and disabling PCIe and USB transactors.

To enable the PCIe or USB transactor, add the following XML configuration:

```
<virtDevHdlPathToDisableWhenActiveHigh>xmr_of_the_signal_used_to_disable</virtDev:HdlPathToDisableWhenActiveHigh>
```

To disable the transactor, ensure that it is done at time 0 before constructing the ProxyEnv.

Support for 10-bit Tag

You can program Device Control 2 register in PCIe Config Space to enable 10 bit tag request. For more information on the register details, refer to the PCIe Spec.

Support for 14-bit Tag

You can program Device Control 3 register in PCIe Config Space to enable 14 bit tag request. For more information on the register details, refer to the PCIe Spec.

PCIe Debug Verbosity

The debug verbosity level in each Test Function can be specified an instantiation time via the `<virtDev:DebugLevel>` XML element. It may also be dynamically modified at runtime by writing the value 0x16C300yz to its Class Code and Revision configuration register at address 0008h.

The debug level of the physical link at the root of the topology might also be dynamically modified at runtime by writing the value 0x16C301yz to the Class Code and Revision configuration register of any function in the PCIe topology. The read-back value of this read-only register is not affected. Here "y" is the debug level of ExpressLink target and "z" is debug level of transactor.

By default, the debug verbosity level is 0 (no debug messages). The larger the debug verbosity level value, the more detailed the debug messages issued by the PCIe Function. Maximum verbosity level is 4.

The debug verbosity level applies to the targeted PCIe Function only. It must be individually set in all or multiple Functions to increase the debug verbosity level in additional Functions.

In XML file, you can add the debug level at different levels like Port level, Device level and Function level:

```
<virtDev:PCIe>
<virtDev:Port>
  <virtDev:Name>link0</virtDev:Name>
  <virtDev:DebugLevel>1</virtDev:DebugLevel>
</virtDev:Port>
<virtDev:Device>
  <virtDev:DebugLevel>1</virtDev:DebugLevel>
  <virtDev:Number>0</virtDev:Number>
  <virtDev:Function>
    <virtDev:Name>NVMe</virtDev:Name>
    <virtDev:DebugLevel>1</virtDev:DebugLevel>
    <virtDev:Number>0</virtDev:Number>
  </virtDev:Function>
</virtDev:Device>
</virtDev:PCIe>
```

Monitoring Performance

Performance metrics can be enabled, displayed, reset, and disabled by writing the value 0x16C302nn to the Class Code and Revision configuration register at address 0008h of any function in the PCIe topology, where "nn" is one of the opcodes detailed in the following table:

Opcode	Description
0h00AA	Start performance measurement. If previous performance metrics were collected, the new metrics are accumulated with the previous ones.
0h00BB	Report current performance measurement statistics.
0x00EE	Stop performance measurement and report accumulated performance statistics
0x00FF	Stop performance measurement, report accumulated performance statistics, then reset accumulated performance statistics.

Performance metrics apply to the link between the PCIe Root Complex and the root of the PCIe device topology, not individual functions or devices in the topology. Writing to the Device ID configuration register of any function in the topology controls performance metrics for the entire topology.

Enabling PCIe Protocol Analyzer From VDM

You can enable the zDPI-based PCIe monitor from VDM. For detailed steps on how to enable and compile the zDPI-based PCIe monitor in ZeBu compilation before enabling from VDM, see the *ZeBu PCIe Transactor User Guide*.

You can enable the Protocol analyzer in XML by adding following tags in the XML configuration file as follows:

```
<virtDev:PCIe>
  <virtDev:Port>
    .
    .
  <virtDev:Debug>
    <virtDev:ProtocolTrace>
      <virtDev:IsEnabled>true</virtDev:IsEnabled>

    <virtDev:HdlInst>wrapper.pcie_wrapper_svs.genblk1.monitor_top_inst_xtor.
xtor_pcie_svs_monitor</virtDev:HdlInst>
    <virtDev:ManualMonitorStart>>false</virtDev:ManualMonitorStart>
  </virtDev:ProtocolTrace>
```

```

        </virtDev:Debug>
        .
        </virtDev:Port>
</virtDev:PCIe>

```

`HdlInst` represents hierarchical path of PCIe monitor in ZeBu compilation. After enabling the analyzer in XML, during the emulation run a `.ztdb` file gets generated with `post_procdumpfile_0.ztdb` in the run area. Follow the post process mechanism defined in the *ZeBu PCIe Transactor User Guide* to convert the `ztdb` file into `fsdb` to view in Verdi protocol analyzer or a text file to view the traces.

In-Band Commands

In-band commands use a regular PCIe transaction to modify or trigger some behavior in VDM. They can be issued by test programs, scripts or command line as required to help debug a problem or create the necessary situation to verify a design. In VDM, all in-band commands are configuration writes of a 4-byte value `0x16C3XXYY` to a VDM function class code register (at address `0x008`). The value "XX" determines the objective of the in-band command and the value "YY" is an objective-specific argument.

Example: `% setpci -s 1:00.0 0x008.l=0x16C30001`

Address	Value	To Function (VDM)
0x008	0x16C300YZ	Set the function's debug level to YZ
	0x16C301YZ	Set debug level of ExpressLink target at the root of the hierarchy ¹ to Z. Set debug level of XTOR at the root of the hierarchy ¹ to Y.
	0x16C302YY	Control collection of PCIe/CXL performance metrics by the ExpressLink target at the root of the hierarchy ¹ , where "YY" is: • "AA": Enable collection. • "BB": Report metrics to date. • "EE": Report metrics to date and disable collection. • "FF": Report and reset metrics, then disable collection.
	0x16C303YY	Trigger an IDE key swap on IDE stream "YY".
	0x16C304YY	N/A
	0x16C30510	Initiate transition to ASPM state L ₁ by the XTOR at the root of the hierarchy ¹ .

Address	Value	To Function (VDM)
	0x16C306YY	Inject AER errors in the function, where "YY" is: • Bit 7 (MSB): Correctable error if set, uncorrectable error if cleared. • Bits 6-5: Reserved. • Bits 4-0: Index of the correctable or uncorrectable error to report. • "FF": Publish AER extended capability with one TLP header record if not already published.
	0x16C307YY	Set debug level for the NVMe Analyzer instance corresponding to this function to "YY". Note that the function must have already been identified as a NVMe device for the debug level setting to take effect.
	0x16C308YY	Start/Stop TLP Recorder ¹ • "00": Start Recording • "01": Stop Recording
	0x16C3FFxx	Internal Notifications – not to be called by an application • "Cy": Notify device at root of topology of APN results, where "y" is a bit mask indicated supported features. "C0" indicates PCIe link. ◦ y[0]: CXL.cache supported ◦ y[1]: CXL.io supported ◦ y[2]: CXL.mem supported ◦ y[3]: Reserved • "FF": Notify device at root of topology that link is ready.

4

CXL

The Virtual Device Model Solution (VDM) enables you to create single-device CXL topologies. CXL switches are not yet supported.

There are two predefined CXL base functions that can be used to create CXL devices. User-defined functions can also be used.

- Type 1 function
- Type 3 function

The CXL topology is specified in the XML file, in a `virtDev:CXL` element.

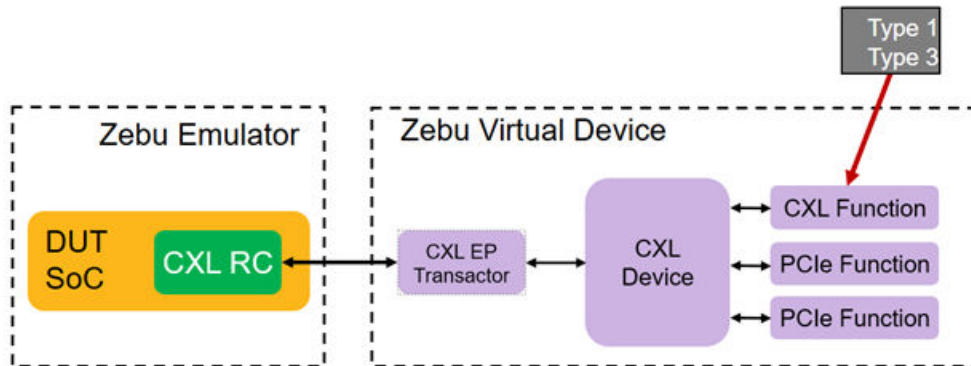
It is the responsibility of the user to correctly enumerate the CXL topology constructed and appropriately assign memory resources. For more details, see *Hardware Abstraction Layer (HAL)* in [PCIe Software Driver](#).

For more information, see the following sections:

- [Single-Device CXL Topology](#)
- [CXL Type 1 Function](#)
- [CXL Type 3 Function](#)
- [CXL Memory Accesses](#)
- [CXL Memory Device Delay Modeling](#)
- [CXL 2.0 Type 1](#)
- [Integrity and Data Encryption for CXL 2.0 Gen5](#)
- [CXL Hot Plug](#)
- [CXL Hot Reset](#)
- [Flit Mode](#)

Single-Device CXL Topology

Figure 8 VDM solution for CXL



Currently, only a single-device CXL topology is supported. As an example, a CXL Type-3 device can be instantiated by defining the following topology:

```

<virtDev:CXL>
  <virtDev:Port>
    <virtDev:Name>link0</virtDev:Name>
    <virtDev:HdlPath>wrapper.cxl_driver_ep</virtDev:HdlPath>
    <virtDev:DebugLevel>0</virtDev:DebugLevel>
    ...
  </virtDev:Port>
  <virtDev:Device>
    <virtDev:DebugLevel>0</virtDev:DebugLevel>
    <virtDev:Function>
      <virtDev:Name>CXL::Type3</virtDev:Name>
      <virtDev:DebugLevel>1</virtDev:DebugLevel>
    </virtDev:Function>
  </virtDev:Device>
</virtDev:CXL>
  
```

A CXL topology is a superset of a PCIe topology, and all XML constructs used to specify a PCIe device can be used to specify the *CXL.io* portion of the CXL device. A CXL device should instantiate a CXL function in slot 0. A CXL device might instantiate PCIe function in additional slots.

CXL Type 1 Function

To instantiate a CXL Type 1 function, specify the name "CXL::Type1" in the `<virtDev::Name>` XML element.

CXL Type 3 Function

To instantiate a CXL Type 3 function, specify the name "CXL::Type3" in the `<virtDev:Name>` XML element.

The following parameters are further configurable using optional XML element should their default not be suitable.

NVMe Parameter	Description	Default
Range1 SizeRange 2 Size	Size of the Range 1 and Range 2 CXL memory decoder, specified as a power of 2. Must be 0, or a value between 28 and 63.	0
Physical memory block.	Lowest physical address in a Device Scoped Memory Affinity Domain (DSMAD) and size of the DSMAD. There can be multiple DSMAD configured in a CXL function, one for each <code><virtDev:DSMAS></code> element. Each creates a DSMAS entry in the Coherent Device Attribute Table (CDAT).	None.

Example XML configuration for a fully configured CXL Type 3 function:

```
<virtDev:Function>
  <virtDev:Name>CXL::Type3</virtDev:Name>
  <virtDev:Number>0</virtDev:Number>
  <virtDev:CXL>
    <virtDev:Range1>
      <virtDev:SizeLog2>28</virtDev:SizeLog2>
    </virtDev:Range1>
    <virtDev:Range2>
      <virtDev:SizeLog2>31</virtDev:SizeLog2>
    </virtDev:Range2>
    <virtDev:CDAT>
      <virtDev:DSMAS>
        <virtDev:SizeLog2>30</virtDev:SizeLog2>
        <virtDev:BaseAddr>0x00000000</virtDev:BaseAddr>
      </virtDev:DSMAS>
      <virtDev:DSMAS>
        <virtDev:SizeLog2>28</virtDev:SizeLog2>
        <virtDev:BaseAddr>0x40000000</virtDev:BaseAddr>
      </virtDev:DSMAS>
      <virtDev:DSMAS>
        <virtDev:SizeLog2>28</virtDev:SizeLog2>
        <virtDev:BaseAddr>0x50000000</virtDev:BaseAddr>
      </virtDev:DSMAS>
    </virtDev:CDAT>
  </virtDev:CXL>
</virtDev:Function>
```

CXL Memory Accesses

QEMU does not model the cache structure of a machine. All memory accesses are performed directly into the memory model. As those accesses are 8 bytes or less, these result in frequent CXL read operations. Only partial writes as CXL memory accesses are based on entire 64 bytes cache lines.

To better model the actual behavior of CXL memory accesses, each CXL Root Port contains ONE cache line. Reading from or writing to a memory region mapped to a CXL memory reads or writes to that cache line. Actual physical CXL memory accesses depend on the state of the accessed memory location in that cache line.

When reading, if the data resides in the cache, the read is completed without any physical accesses. If the data does not reside in the cache, a physical CXL memory read is performed to fetch the entire line that contains the data read and merged with any previously written (“dirty”) bytes. If the cache line was “dirty” and contained data from a different cache line, a physical CXL memory write is performed first to write back the updated cache line data into the CXL memory.

When writing, if the cache line is empty or the data already resides in the cache, the content of the cache is updated, and the write operation is completed without any physical accesses. If the data does not reside in the cache and the current content of the cache line is “dirty”, a physical CXL memory write is performed first to write back the updated cache line data into the CXL memory before flushing the cache line and writing the updated data into the now-empty cache. The content of the cache line is not fetched. If a write access is performed to the last bytes of the cache line and all of the previous bytes are “dirty”, the content of the cache line is written through to the CXL memory.

CXL Memory Device Delay Modeling

By default, CXL.mem requests are fulfilled, from M2S requests to S2M response, by the function in zero-time. Any delay in the response is introduced by the physical CXL link and the time required by the transactor to receive and process the request, and the time required by the transactor to process and transmit the response, which might include additional waiting time queued for a transmission opportunity.

Whenever a M2S request is received by the CXL function, a user-defined (or predefined) response delay model C++ function is called to determine the delay to insert before the S2M response is sent. The delay is nonblocking, meaning that a subsequent M2S request can be received while the response for the previous one is still waiting to be sent. The delay in responding to that subsequent request might even be less than the remaining delay on the first request, meaning that delayed responses could be sent out of order. Note that the response delay value only models the function response delay before it is submitted to the transactor to be physically transmitted to the requester. The actual response delay also includes transactor and link delays and might be longer.

Predefined Response Delay Model Functions

A number of predefined delay model functions are available. They can be selected by name in the `virtDev::MemoryResponse/virtDev:DelayFunction` element, followed by parameter values that override the default characteristics of the predefined function.

```
<virtDev:Function>
  <virtDev:Name>CXL::Type3</virtDev:Name>
  <virtDev:MemoryResponse>
    <virtDev::DelayFunction>functionName</virtDev:DelayFunction>
    <virtDev:Param>...</virtDev:Param>
    <virtDev:Param>...</virtDev:Param>
    ...
  </virtDev:MemoryResponse>
</virtDev:Function>
```

The following predefined delay model functions are available:

Name	Description		
constant	The response delay for a request is constant, independently of previous requests. Address, rate, and queue length are ignored.		
	Parameters		
	param[0]	delay	Response delay. Default is 0.
exponential	The response delay is computed as $D + E(rate - Tr)$. See https://www.desmos.com/calculator/1c6bgs7qzk for an interactive plot of the delay function and helps determine the required parameter values.		
	Parameters		
	param[0]	D	Minimum response delay. Default is 0.
	param[1]	Tr	Arrival rate threshold. Default is 100.
	param[2]	E	Exponent base. Must be greater than or equal to 1. Default is 1.05. Note: Delay is constant and equal to D if equal to 1.

User-Defined Response Delay Model Functions

User-defined response delay model functions are functions with the following signature and return the number of CXL clock cycles to delay the response as an unsigned 16-

bit number. All time values are expressed as "number of PCIe/CXL clock cycles", which translates to 1ns units.

```
uint16_t delayFct(SNPS::VSA::VDM::Utils::Logger &log,
                  bool isRead,
                  uint64_t addr,
                  uint16_t rate,
                  const std::vector<float> &params,
                  void* &state);
```

Argument Name	Description
log	Reference to the function's logger instance
isRead	TRUE if the access is a read access.
addr	Address of the requested memory.
rate	Time elapsed since the previous memory request. Can be used to determine the instantaneous bandwidth utilization. For the very first request and very large time intervals, this value is saturated to 0xFFFF.
params	User-specified parameters
state	Optional opaque user-defined state information

An optional opaque pointer, by default set to NULL, can be used to pass additional context information or maintain state information across invocations. The following example shows how a random-stable delay model function could be implemented.

```
uint16_t stableFct(...)
{
    auto myState = (myStateTyp*) state;
    if (myState == NULL) {
        state = (myState = new myStateTyp);
        myState->rnd = new std::default_random_engine;
    }

    std::uniform_int_distribution<uint16_t> dist(...);
    return dist(myState->rng);
}
```

In addition to the opaque user-defined pointer, delay model functions can also be supplied with a variable number of floating-point parameter values. These can be used to modify the delay model parameters from run to run via the topology XML file in the virtDev::CXL element inside the corresponding virtDev::Function.

Upon registration, the delay model function is immediately called with an address value of 0xFFFFFFFFFFFFFFFF, a rate value of 0xFFFF, and user-defined parameter values (if any). The return value is ignored. It is an opportunity for the function's implementation to issue a one-time debug message describing its functionality, parameter semantics, and parameter values.

```
uint16_t exponentialFct(...)
{
    uint16_t D = (param.size() >= 1) ? param[0] : 0;
    uint16_t T = (param.size() >= 2) ? param[1] : 100;
    double E = (param.size() >= 3) ? param[2] : 1.05;

    if (addr == 0xFFFFFFFFFFFFFFFF && rate == 0xFFFF) {
        log.debug(0, "Using 'D+E^(rate-T)' response delay model");
        log.debug(0, "    param[0]: D = %d", D);
        log.debug(0, "    param[1]: T = %d", T);
        log.debug(0, "    param[2]: E = %.4e", E);

        return 0;
    }
    ...
}
```

User-defined response delay model function implementations can be assigned to a CXL function instance procedurally, or via a shared library and XML configuration.

Using the procedural method, the response delay function is usually implemented in the C++ testbench instantiating the Virtual Device environment, and is attached to the CXL function using the *SNPS::VSA::VDM::CXL::Function::defineDelayModel()* method.

```
void defineDelay(SNPS::VSA::VDM::CXL::Function::delayFct_t fct,
                void* state = NULL);
```

When specifying the response delay model via XML configuration, the delay function must be compiled into an object file linked into the final testbench binary, and the name of the delay function specified using the *<virtDev:DelayFunction>* XML element.

```
<virtDev:Function>
    <virtDev:Name>CXL::Type3</virtDev:Name>
    <virtDev:MemoryResponse>
        <virtDev:DelayFunction>myDelayFct</virtDev:DelayFunction>
        ...
    </virtDev:MemoryResponse>
</virtDev:Function>
```

Note:

The delay function is not specified with "C" linkage using the "extern "C"" attribute. The name of the delay function is suitably mangled into its C++ symbol name equivalent when calling it to generate the next delay.

CXL 2.0 Type 1

VDM supports the CXL 2.0 Type 1 topology. To use the feature, first set the variable `USE_TYPE1_VDM_WL` to 1. For detailed steps on how to compile the design using this feature, refer to the `readme` file available at `Examples/VHS_VDM/zRci/runtime/README`.

- To run VDM (with CXL-TYPE1 Workload) use below command with `USE_TYPE1_VDM_WL` set to 1.

```
% make run_zrci USE_VHS=0 USE_VDM=1 USE_TYPE1_VDM_WL=1
```

- To run VHS+VDM with (CXL-TYPE1 Example) use below command:

```
% make run_zrci CONFIG_XML=config_vhs_vdm_cxl_type1.xml USE_VHS=1  
USE_VDM=1 USE_TYPE1_VHS_RSP=1 USE_TYPE1_VDM_WL=1
```

Integrity and Data Encryption for CXL 2.0 Gen5

To use Integrity and Data Encryption (IDE), you need a special configuration of the CXL transactor. The traffic between a Virtual Host Root Complex (RC) and a PCIe function is not encrypted by default. To encrypt some or all of the traffic between the RC and a DUT, you need to integrate the transactor with the Synopsys Secure IP. You also need to ensure that IDE is enabled in the XML configuration specification of the Root Complex.

```
<virtDev:IDE>  
    <virtDev:IsEnabled>true</virtDev:IsEnabled>  
</virtDev:IDE>
```

Pre-Shared Keys

Using pre-shared keys allows IDE streams to be configured and secured without any specialized driver code executing on the host. To enable IDE and use Pre-Shared Keys (PSK), set the tags as shown in the following XML specification:

```
<virtDev:IDE>
  <virtDev:IsEnabled>true</virtDev:IsEnabled>
  <virtDev:PreSharedKeys>
    <virtDev:StreamType>Link</virtDev:StreamType>
    <virtDev:AutoSecure>true</virtDev:AutoSecure>
    <virtDev:StreamId>1</virtDev:StreamId>
    <virtDev:TrafficClass>7</virtDev:TrafficClass>
    <virtDev:KeySet>
      <virtDev:PR>
        <virtDev:RxKey>000102030405060708090a0b0c0d0e0f202122232425262728292a2b2c2d2e2f</virtDev:RxKey>
        <virtDev:RxIV>7071727374757677</virtDev:RxIV>
        <virtDev:TxKey>a0a1a2a3a4a5a6a7a8a9aaabacadaeaf808182838485868788898a8b8c8d8e8f</virtDev:TxKey>
        <virtDev:TxIV>f0f1f2f3f4f5f6f7</virtDev:TxIV>
      </virtDev:PR>
      <virtDev:NPR>
        <virtDev:RxKey>101112131415161718191a1b1c1d1e1f303132333435363738393a3b3c3d3e3f</virtDev:RxKey>
        <virtDev:RxIV>d0d1d2d3d4d5d6d7</virtDev:RxIV>
        <virtDev:TxKey>b0b1b2b3b4b5b6b7b8b9babbbcbdbef909192939495969798999a9b9c9d9e9f</virtDev:TxKey>
        <virtDev:TxIV>e0e1e2e3e4e5e6e7</virtDev:TxIV>
      </virtDev:NPR>
      <virtDev:CPL>
        <virtDev:RxKey>202122232425262728292a2b2c2d2e2f404142434445464748494a4b4c4d4e4f</virtDev:RxKey>
        <virtDev:RxIV>5051525354555657</virtDev:RxIV>
        <virtDev:TxKey>c0c1c2c3c4c5c6c7c8c9cacbcccdcecf0a0a1a2a3a4a5a6a7a8a9aaabacadaeaf</virtDev:TxKey>
        <virtDev:TxIV>d0d1d2d3d4d5d6d7</virtDev:TxIV>
      </virtDev:CPL>
      <virtDev:CP>
        <virtDev:RxKey>303132333435363738393a3b3c3d3e3f404142434445464748494a4b4c4d4e4f</virtDev:RxKey>
        <virtDev:RxIV>4041424344444647</virtDev:RxIV>
        <virtDev:TxKey>d0d1d2d3d4d5d6d7d8d9dadbdbdddedfa0a0a1a2a3a4a5a6a7a8a9aaabacadaeaf</virtDev:TxKey>
        <virtDev:TxIV>c0c1c2c3c4c5c6c7</virtDev:TxIV>
      </virtDev:CP>
    </virtDev:KeySet>
  </virtDev:PreSharedKeys>
</virtDev:IDE>
```

Each IDE stream must be configured with sets of Tx and Rx key that match the Rx and TX keys configured in the peer endpoint function (host side to device side. Keys are specified as sets of 256-bit AES keys and 64-bit Initial Values (IV), one for each direction in each sub stream.

Note the following tags:

- `<virtDev:IsEnabled>` tag is set to true
- `<virtDev:StreamType>` tag is used to configure the IDE stream. To configure CM keys, use only the Link streamtype.
- `<virtDev:StreamId>` and `<virtDev:TrafficClass>` tags are used to specify a unique stream ID and a traffic class. Note that in case CM keys are used, set the stream ID to 1.

CXL Hot Plug

VDM supports the CXL hot plug feature. CXL hot plug allows an endpoint to be inserted or removed from CXL without special consideration.

In the case of emulation, these events can typically be mimicked by putting the device under reset (Plug-out/unplugged state) and bringing the device out of reset (Plug-in/Plugged state).

In VDM, hot plug can be achieved by applying dynamic reset from the RC transactor side. This provides a behavior that is comparable to Hot-Plug.

Note:

You can access Root-Port/Bridge/RC.

When the user brings the EP from reset, link-training, speed up, and device enumeration happens automatically, and you can access the EP device.

Note:

This support is available only with a DPI transactor.

CXL Hot Reset

VDM supports hot reset feature. Hot-Reset is a mechanism to reset the CXL device by programming the Root port's reset bit of the bridge control register. For more information, refer to the *ZeBu CXL Transactor User Manual*.

Flit Mode

CXL VDM supports all Flit modes and speeds mentioned in the CXL specifications table shown in the following figure:

256B Flit Mode vs. 68B Flit Mode Operation

Data Rate	PCIe Flit Mode	Encoding	CXL Flit Mode
2.5 GT/s, 5 GT/s	No	8b/10b	CXL is not supported
2.5 GT/s, 5 GT/s	Yes	8b/10b	CXL is not supported
8 GT/s, 16 GT/s, 32 GT/s	No	128b/130b	68B flits
8 GT/s, 16 GT/s, 32 GT/s	Yes	128b/130b	256B flits
64 GT/s	Yes	1b/1b	256B flits

Using CXL VDM with CXL 2.0 Mode

To use CXL VDM with CXL 2.0 mode, configure the `<virtDev:Version>` tag in the VDM XML file as follows:

```
<virtDev:Version>2.0</virtDev:Version>
```

To specify different generation speeds, configure the `<virtDev:TargetSpeed>` tag with the required generation speed as follows:

For Gen3 speed, specify `<virtDev:TargetSpeed>3</virtDev:TargetSpeed>`

For Gen4 speed, specify `<virtDev:TargetSpeed>4</virtDev:TargetSpeed>`

Gen5 is the default speed with CXL 2.0.

Using CXL VDM with CXL 3.0 (256B Flit) Mode

To use CXL VDM with CXL 3.0 (256B Flit) mode, configure the `<virtDev:Version>` tag in the VDM XML file as follows:

```
<virtDev:Version>3.0</virtDev:Version>
```

To use different generation speeds, configure the `<virtDev:TargetSpeed>` tag with the required generation speed as follows:

For Gen3 speed, specify `<virtDev:TargetSpeed>3</virtDev:TargetSpeed>`

For Gen4 speed, specify `<virtDev:TargetSpeed>4</virtDev:TargetSpeed>`

For Gen5 speed, specify `<virtDev:TargetSpeed>5</virtDev:TargetSpeed>`

Gen6 is the default speed with CXL 3.0 (256B Flit) mode.

Using CXL VDM with CXL 3.0 (68B Flit) Mode

Finally, to use CXL VDM operation in **CXL 3.0 (68B Flit) mode**, configure the `<virtDev:Version>` and `<virtDev:FlitMode>` tags in the VDM XML file as follows:

```
<virtDev:Version>3.0</virtDev:Version>  
<virtDev:FlitMode>68</virtDev:FlitMode>
```

To use different generation speeds, configure the `<virtDev:TargetSpeed>` tag with the required generation speed as follows:

For Gen3 speed, specify `<virtDev:TargetSpeed>3</virtDev:TargetSpeed>`

For Gen4 speed, specify `<virtDev:TargetSpeed>4</virtDev:TargetSpeed>`

Gen5 is the default speed with CXL 3.0 (68B Flit) mode.

5

USB

USB VDM supports USB 2.0 (FS or HS) and USB 3.1 (SS or SSP) speeds. USB mass storage model speeds can be configured using the `Create()` method.

Syntax

```
void Create(const std::string& device_speed);
```

where, possible values of `device_speed` variable are as follows:

- `super_plus_speed`
- `super_speed`
- `high_speed`
- `full_speed`

Example to use create method

```
#if defined(USB_MASS_STORAGE_MODEL)
    //Lets create the USB Device Model...
    dev_model = new USB3DeviceMassStorage(
        dev_inst_1, "hw_top.top1.clk_48m",
        "usb_strick_1.img", 0xA00000) ;
    assert (dev_model != nullptr) ;
    dev_model->Create("super_plus_speed") ;
    dev_model->setXtorDebug("USB_INST_1", 1) ;
    dev_model->setDebug("VDM_1", 1) ;
#endif
```

6

Examples

This chapter provides instructions for running PCIe, Virtual NVMe, USB, ENET, and UFS examples.

It consists of the following topics:

- [Settings for Running an Example](#)
- [PCIe Example](#)
- [CXL Example](#)
- [USB Example](#)

Settings for Running an Example

Environment Settings

For running VSA examples, you need to set the following mandatory environment settings. Before you begin with the settings, set `VSA_ABI` as specified in the ABI Support subsection of the ABI Support section.

```
% setenv ZEBU_ROOT <PATH_TO_ZEBU_ROOT>
% source ${ZEBU_ROOT}/zebu_env.csh
% setenv ZEBU_IP_ROOT <PATH_TO_ZEBU_IP_ROOT>
% setenv LD_LIBRARY_PATH
${ZEBU_IP_ROOT}/libABI${VSA_ABI}:${LD_LIBRARY_PATH}
% setenv PATH ${ZEBU_IP_ROOT}/binABI${VSA_ABI}:${PATH}
% setenv LD_LIBRARY_PATH <PATH_TO_GCC>/lib64:${LD_LIBRARY_PATH}
% setenv PATH <PATH_TO_GCC>/bin:${PATH}
% setenv VSA_VDM_ROOT
% source ${VSA_VHS_ROOT}/setup.csh
```

Testbench Location

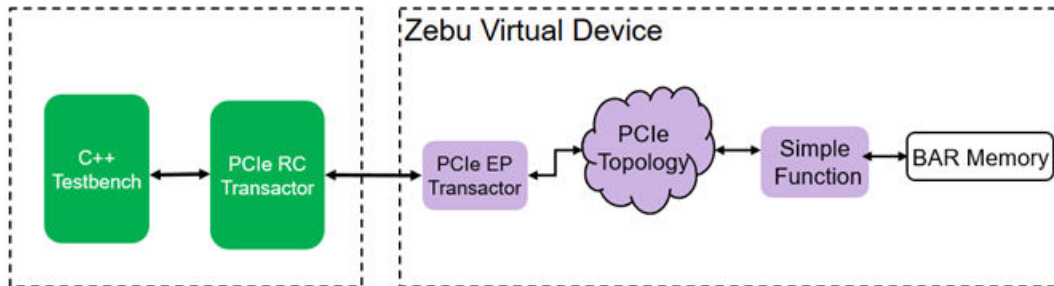
VHS supports two types of testbench formats: C++ based and zRci based. Depending on the type of testbench you are using, you can go to the respective Examples folder under `$VSA_VDM_ROOT/Examples/<Protocol>`.

For example, to access the PCIe example, go to `Examples/PCIe`.

PCIe Example

In the following example, VHS and VDM are connected back-to-back.

Figure 9 VDM PCIe using Simple function to access the memory



Compile

To compile the PCIe example, follow the instructions in the transactor examples area. After you have a successful ZeBu compile, update the compiled database path in VDM XML file as following:

```
<vHost:ZebuWorkPath>/myopath/16x32b_b2b.zebu.work/zebu.work</vHost:ZebuWorkPath>
```

Both root complex (RC) and end point (EP) as pcie transactors are connected back to back.

```
Testbench <-> Xtor(RC) <-> Lane Model <-> Xtor (EP) <-> PCIe EP Model  
(Simple Function)
```

Runtime

Update XML/OneTester.xml on the type of PCIe Device function that needs to be used. In this example, Simple Function is used. For details on configuring the Simple Function, see [Simple Function](#).

Src/bench/testbench.cc is used to configure the RC XTOR, link speeds as well as the test sequences.

Current testbench completes linkup along with Enumeration and few CfgRd and CfgWr sequences.

1. Make sure the `zebuwork` is available in the ZeBu folder.
2. Run test.

CXL Example

This CXL example runs with VHS and VDM packages. Currently it is supported in VHS-VDM connected in back-to-back mode only.

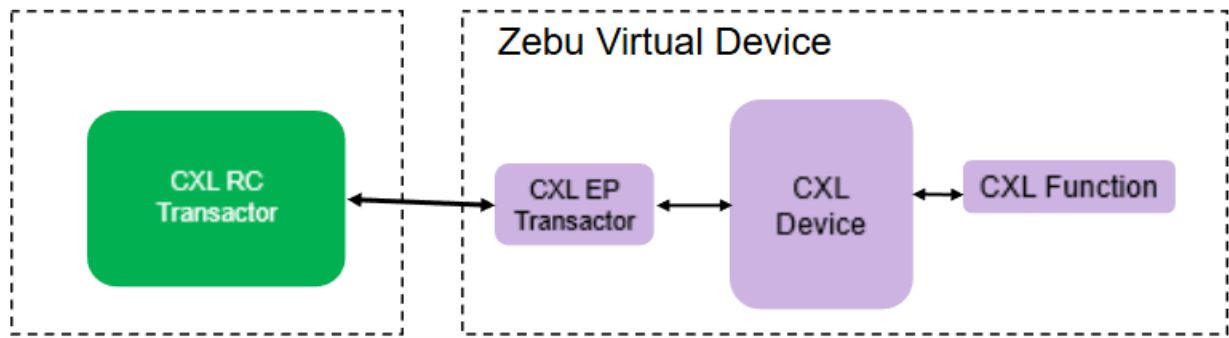
Note:

For running this example, you must have VSA_VHS license.

This example setup is available in the following directory:

`<VSA-VDM Package>/Examples/CXL`

Figure 10 VDM CXL using Simple function to access the memory



Perform the following steps to run the CXL Typ3 device example:

1. Edit `<SandBox_Name>/VHS_VDM/shared/config/config_vhs_vdm_cxl_type3.xml` to update `zebu.work` and `designFeature` file path and change CXL Host as required.
2. In the XML under VHS ProxYXtor section, the `<vHost:Type>CXL30</vHost:Type>` represents the VHS as a CXL3.0 Host. If you want to run as CXL 2.0 Host, change Type to field to CXL20.
3. CXL version in VDM XML can be set under number `<virtDev:Version>3.0</virtDev:Version>` 3.0 represent a CXL 3.0 Device. 2.0 represents the CXL 2.0 compliant device.

The run command should do the following:

1. Launch the ZeBu
2. Wait for Link up and enumeration to finish
3. Create Socket File

```
% make run_zrci CONFIG_XML=config_vhs_vdm_cxl_type3.xml USE_VHS=1  
USE_VDM=1
```

To Run Virtual Host to Enumerate VDM Model (Using B2B ZeBu.Work), update
<SandBox_Name>/PCIE/shared/Config.mk file with correct VM File path (QCOW2_IMG
path)

```
cd <SandBox_Name>/VHS_VDM/zRci/runtime  
  
% make ProxyPingQemu CONFIG_XML=config_vhs_vdm_cxl_type3.xml  
  
% make QemuRun CONFIG_XML=config_vhs_vdm_cxl_type3.xml QCOW2_IMG=<Path to  
your QCOW2 image file>
```

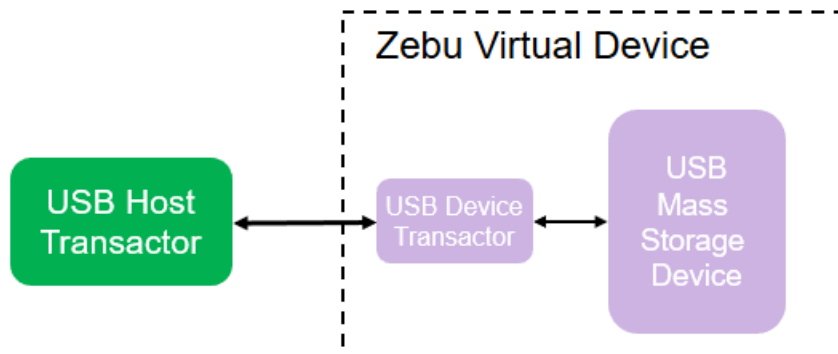
USB Example

USB example is run using both VHS and VDM packages. Currently it is supported in VHS-VDM Transactor connected in back-to-back mode only.

Note:

VDM only mode is currently not supported. Only USB3.1 Spec flow supported.

Figure 11 VDM Solution for USB



To compile and run the USB example, perform the following steps:

1. Copy the example source file in a local directory and ensure that directory is the current working directory.

```
% cp -r $VSA_VDM_ROOT/Examples/USB3xDPI somewhere/local  
% cd somewhere/local/USB3xDPI
```

2. Compile the Verilog code for the transactor by following the steps specified in the USB Transactor Example setup.
3. Compile the ZeBu proxy transactor testbench server.

```
% make zrci
```

4. Use the CONFIG_XML file corresponding to your configuration and update zebu.work path, designFeatures, and other configurations.

Note:

Use the XML file - config.xml for 1 port configuration.

5. Use the following XML file - config_two_usb_devs.xml for 2 port configurations. Run the test case on the emulator:

- a. Single USB VHS Host and Single VDM device

```
% make run_zrci USB_MASS_STORAGE_MODEL=1
```

- b. Two USB VHS hosts and two VDM devices (required NUM_INST=2 compile)

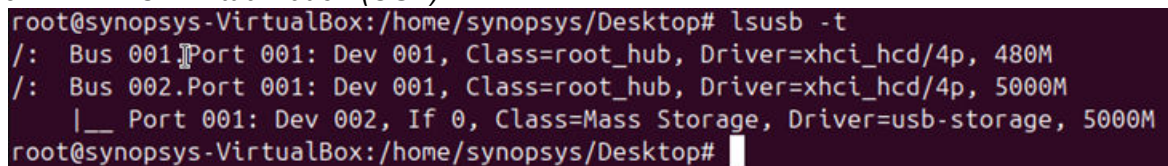
```
% make run_zrci USB_MASS_STORAGE_TWO_INSTANCES=1
```

6. Start the VM after you see the *waiting for client to connect* message.

```
make QemuRun QARGS=<Path to qcow image file> USE_XML_FILE=<name of XML used>
```

7. Ensure that the DUT is virtualized and associated with the correct USB host controller by using the lsusb and lsusb -t commands, as follows:

Figure 12 DUT Virtualization (USB)



```
root@synopsys-VirtualBox:/home/synopsys/Desktop# lsusb -t
/: Bus 001.Port 001: Dev 001, Class=root_hub, Driver=xhci_hcd/4p, 480M
/: Bus 002.Port 001: Dev 001, Class=root_hub, Driver=xhci_hcd/4p, 5000M
   |__ Port 001: Dev 002, If 0, Class=Mass Storage, Driver=usb-storage, 5000M
root@synopsys-VirtualBox:/home/synopsys/Desktop#
```

7

Appendix: Verified Software Drivers

There is often a complex discovery and configuration process involved in getting devices ready to be used. If the Virtual Device Model Solution (VDM) is used in combination with a host running a standard operating system, that discovery and configuration process is handled by the BIOS or the operating system. But when used in a standalone or bare-metal environment, it is still necessary to perform that discover and configuration process.

If the verification objective is not to verify that response of the device to that discovery and configuration process, or to verify its response to erroneous configuration, VDM provides pre-compiled discovery and configuration "drivers", as well as some protocol stacks.

The C++ header files defining these predefined software driver components can be found in the `$VSA_VDM_ROOT/drivers/include` directory. The pre-compiled object files for the x86 and ARM64 architecture can be found in the `$VSA_VDM_ROOT/drivers/libABI?/x86` and `$VSA_VDM_ROOT/drivers/libABI?/aarch64` directories, respectively.

Detailed documentation of the various APIs can be found in the Doxygen output included in the package.

PCIe Software Driver

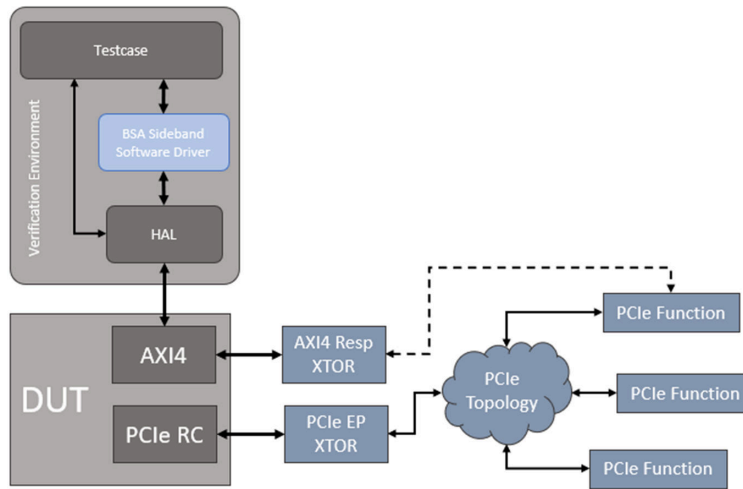
The PCIe software driver provides a function to enumerate and configure the entire PCIe topology found less than a specified bus number. A single instance of the `SNPS::VSA::VDM::SW::PCIe::Driver` per host is sufficient. It also allocates MMIO resources starting from user-specified 32-bit and 64-bit addresses. For each device discovered and configured, it invokes a callback with a descriptor providing detailed probing information on the newly-discovered function.

For more details, see the `SNPS::VSA::VDM::SW::PCIe::Driver` class.

Hardware Abstraction Layer

The PCIe software driver performs its task by executing configuration and memory read/write transactions targeting the various PCIe devices it discovers. As the means of doing so are different from verification environment to verification environment and from host model to host model, those operations are performed through a Hardware Abstraction Layer (HAL) that must be implemented by the user.

Figure 13 PCIe Hardware Abstraction Layer



It is recommended that this software driver be instantiated even if it is not used for final PCIe device enumeration and configuration, as it is used as the HAL for higher-level protocol stacks, therefore eliminating the need for implementing further HAL APIs. Further, the entire HAL API is reflected as publicly accessible methods in the Driver class. It is a good practice for the test case needing to perform similar operations to use them to minimize maintenance and increase test case portability, as shown in the following example.

```
#include <PCIEDriver.hh>

SNPS::VSA::VDM::Utils::Logger    log("test");
SNPS::VSA::VDM::SW::PCIe::Driver pcieDrvr(log);

std::map<uint16_t id, const SNPS::VSA::VDM::SW::PCe::FunctionInfo>
functionsById;

void discoveryFct(const SNPS::VSA::VDM::SW::PCe::FunctionInfo &fct, void*
context)
{
    functionsById[fct.bdf.toID()] = fct;
}
...
int main(...)
{
    ...
    uint8_t bus    = 1;
    uint16_t MPS   = 4096;
    uint16_t MRRS  = 4096;

    SNPS::VSA::VDM::SW::PCIe::Driver::MemAlloc memMap;
    memMap.memBase32 = 0xC0000000;
```

```
memMap.memBase64 = 0x000000001000000000;  
  
pcieDrvr.enumerateTopology(bus, MPS, MRRS, memMap, discoveryFct);  
...  
uint32_t IDs;  
pcieDrvr.cfgRd(1, 0, 0, 0x000, 4, IDs);  
...  
}
```

The entire HAL that must be contained within the `SNPS::VSA::VDM::SW::PCIE::Driver::HAL` naming scope. For more information, see the Doxygen output included in the package.

Every HAL API required by the PCIe software driver is specified as a static method, but no implementation is supplied. It must be supplied by the user and can be implemented as a C++ global function in the verification environment.

```
#include <PCIEDriver.hh>  
  
SNPS::VSA::VDM::PCIE::Status_t  
SNPS::VSA::VDM::SW::PCIE::Driver::HAL::cfgRd(void* context, uint8_t  
busNum, uint8_t devNum, uint8_t fnNum, uint16_t addr, unsigned char  
bytes, uint32_t &rdat, Attributes *attr)  
{  
    rdat = ...;  
    return SNPS::VSA::VDM::PCIE::SUCCESS;  
}
```

An optional user-defined opaque context pointer, specified when instantiating the `SNPS::VSA::VDM::SW::PCIE::Driver` class, is passed to every HAL function called by that instance. This provides a mechanism for distinguishing the instance of the PCIe software driver that is using the HAL, should multiple instances exist in a verification environment.

CXL Software Driver

The CXL software driver is an extension of the PCIe software driver and provides additional CXL-specific configuration and convenience functions. A single instance of the `SNPS::VSA::VDM::SW::CXL::Driver` per host is sufficient and can be used to access PCIe ports as well as CXL ports.

For more details, see the `SNPS::VSA::VDM::SW::CXL::Driver` class.

Hardware Abstraction Layer (HAL)

The CXL software driver uses the same Hardware Abstraction Layer as the PCIe software driver and does not require any additional HAL implementations.

DOE Software Driver

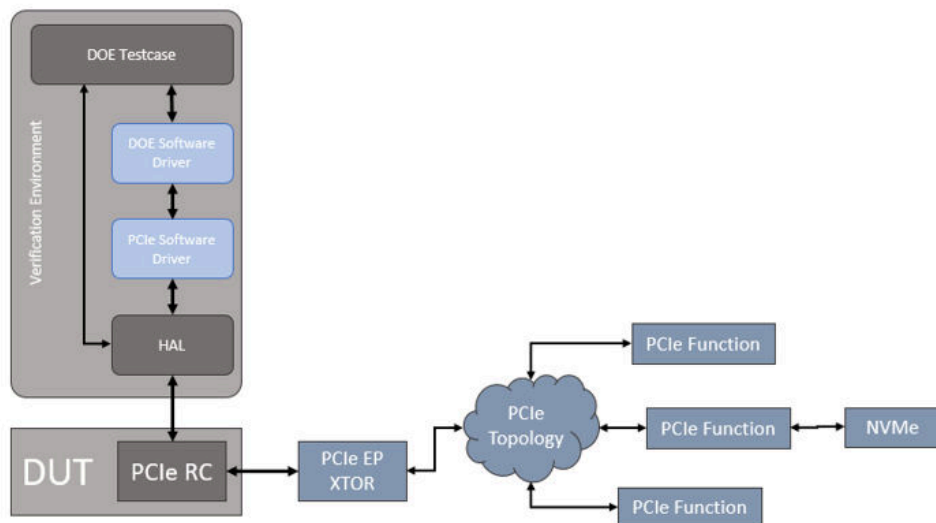
The DOE software driver provides a set of convenience functions to send requests and received responses from a Data Object Exchange capability in a PCIe function. There must be one instance of the `SNPS::VSA::VDM::SW::PCIe::DOE::Driver` per DOE capability instance, whether in the same or different PCIe functions. Each instance is constructed to operate via a specific PCIe software driver, targeting a DOE Extended Capability instance located at a specified configuration address offset in a specified PCIe function.

For more details, see the `SNPS::VSA::VDM::SW::PCIe::DOE::Driver` class.

Hardware Abstraction Layer

The DOE software driver performs its tasks by executing configuration read and write transactions to the specified DOE Extended Capability registers in the specified function using the specified PCIe software driver. It does not require any additional HAL API implementations.

Figure 14 DOE Topology



NIMG File Format

NIMG files are binary files composed of a header specifying the format of the storage blocks and metadata, followed by the block data and block metadata in consecutive order.

If a file does not contain the data for a specific block, it is assumed to be 0s. The block size and metadata size cannot be modified without completely rewriting the file.

Header Format

The NIMG file header starts at offset 0h and contains the following values in little-endian order. The values must be valid values according to the NVMe specifications.

Offset	Size	Description
00h	4	Magic value 0x16C35503
04h	4	Namespace size (NSZE), in number of blocks
08h	4	Logic block data size (LBADS), in number of bytes
0Ch	2	Metadata size (MS), in number of bytes
0Eh	1	Metadata Settings (MSET). Currently, only '1' is supported.
0Fh	1	Reserved
10h	16	Reserved

Block Format

Block #0 immediately follows the header. The block data is contained in the next “LBADS” number of bytes, followed by “MS” number of metadata bytes (if present). The next block starts immediately after the last metadata byte of the previous block.

Offset	Size	Description
Block Number * (LBADS + MS) + 20h	LBADS	Block data
Block Number * (LBADS + MS) + LBADS + 20h	MS	Metadata
(Block Number + 1) * (LBADS + MS) + 20h	LBADS	Block data

The total number of blocks in a file is determined by the file size. The maximum number of blocks in a file is limited by the size of the NVMe namespace and the file may grow to accommodate the number of blocks written in a namespace.